Theoretical Framework: Generative Surrogates for PDE Solution Fields

Project: laplace-uq-bench **Author:** Jane Yeung **Date:** March 2026 **Companion to:** `laplace-uq-bench/` implementation plan **Python package:** `diffphys` (code paths use `src/diffphys/` throughout; the repository was renamed but the Python package name is retained for import stability)

Table of Contents

1. Motivation & Physical Setup
2. The Forward Problem: Laplace's Equation
3. Neural Surrogate Architectures
4. Score-Based Diffusion and Langevin Dynamics
5. Posterior Inference Under Uncertain Observations
6. Physics-Informed Regularization
7. Conditional Flow Matching and Optimal Transport
8. Improved DDPM Training
9. Conformal Prediction for Calibrated Uncertainty
10. Diffusion Posterior Sampling
11. Evaluation Theory: Proper Scoring and Calibration
12. Reference Paper Connections
13. Theory-to-Deliverable Mapping
14. Validation Criteria
15. Notation Reference
16. Bibliography

---

1. Motivation & Physical Setup

1.1 The Physical System

We study the **steady-state heat equation** (Laplace's equation) on the unit square $\Omega = [0,1]^2$ with Dirichlet boundary conditions:

$$\nabla^2 T(x,y) = \frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0 \quad \text{on } \Omega \tag{1.1}$$

$$T = g(x,y) \quad \text{on } \partial\Omega \tag{1.2}$$

where $T(x,y)$ is the temperature field and $g(x,y)$ prescribes the boundary profile. Laplace's equation arises as the equilibrium limit of the heat equation $\partial_t T = \kappa \nabla^2 T$ — when all transient behavior

has decayed, the temperature distribution satisfies (1.1). It is the prototypical second-order elliptic PDE: linear, self-adjoint, and admits a unique classical solution for any continuous Dirichlet data g on a bounded domain with smooth boundary.

1.2 Why This System

Laplace's equation is chosen *precisely because it is simple*. The PDE has a known, fast numerical solver — a finite-difference discretization with LU factorization solves each instance in $O(1)$ ms. Neural surrogates cannot beat this solver on accuracy or speed for this problem.

The simplicity is the point. Because the numerical oracle is cheap and its truncation error is negligible for smooth BCs at $N = 64$ (see §2.2), we can: 1. Generate arbitrarily large datasets with negligible solver error. 2. Evaluate neural surrogates against an oracle whose solutions are accurate to well below the resolution any neural surrogate will achieve. 3. Isolate the question "when does a generative model add value?" from confounding factors like solver error, mesh dependence, or turbulence modeling.

This is a **controlled experimental setting** for studying neural surrogate uncertainty, not a claim that diffusion models should replace PDE solvers.

1.3 Why These Computational Methods

The project proceeds in two phases. **Phase 1** establishes deterministic baselines and the DDPM generative framework; **Phase 2** compares three approaches to uncertainty quantification under noisy boundary observations, with a fourth as a stretch goal. Sections 1–6 cover the PDE setup, baseline architectures, DDPM theory, and evaluation foundations shared by both phases; §7–§10 extend the project with the Phase 2 methods: flow matching, improved DDPM training, conformal prediction, and DPS.

Method	Type	Uncertainty?	Why included	Phase
U-Net regressor	Deterministic	No	Simplest deep learning baseline	1
FNO	Deterministic	No	Operator-learning inductive bias (underperformed; see §3.2)	1
Deep ensemble (5×)	Probabilistic, non-generative	Epistemic only	Calibration baseline	1–2
Ensemble + conformal	Probabilistic, non-generative	Post-hoc calibration wrapper	Coverage calibration at zero training cost	2
Conditional DDPM (improved)	Probabilistic, generative	Approximate posterior samples	Best functional CRPS on this benchmark (sparse-noisy, matched 5v5)	2

Method	Type	Uncertainty?	Why included	Phase
Conditional flow matching	Probabilistic, generative	Approximate posterior samples	Alternative generative framework	2
DPS (unconditional + guidance)	Probabilistic, generative	Approximate posterior samples	Stretch goal: zero-shot adaptation	2

The Phase 2 comparison tests three UQ approaches: **(1)** improved DDPM (§8) with cosine schedule, v-prediction, and Min-SNR weighting, which achieved the best functional CRPS on this benchmark (sparse-noisy regime, matched 5v5) and tightest pixelwise conformal intervals; **(2)** conditional flow matching (§7), which offered simpler training dynamics but underperformed DDPM on most metrics in this setting — suggesting that on this problem, the training improvements (§8) were more impactful than the choice of generative framework; and **(3)** deep ensembles with pixelwise conformal prediction (§9), a practical post-hoc calibration wrapper; the exact finite-sample coverage guarantee applies to standard split conformal prediction (§9.1, Eq. 9.4), while the pooled-pixel variant used in the benchmark is an empirical adaptation (see §9.1 exchangeability caveat). DPS (§10) is a stretch goal exploring inference-time adaptation.

All models are implemented in `src/diffphys/model/`*. The U-Net backbone (*`model/unet.py`*) is shared across the regressor, ensemble members, DDPM, and flow matching. The FNO (*`model/fno.py`*) uses independent architecture. All conditional models consume the 8-channel conditioning tensor defined in §2.5 and produced by* `src/diffphys/data/conditioning.py`*.*

---

2. The Forward Problem: Laplace's Equation

2.1 Existence, Uniqueness, and Maximum Principle

Theorem (Existence and Uniqueness). For continuous Dirichlet data $g \in C(\partial\Omega)$ on a bounded domain $\Omega \subset \mathbb{R}^2$ with smooth boundary, there exists a unique solution $T \in C^2(\Omega) \cap C(\bar{\Omega})$ to (1.1)–(1.2). [Exact; see Evans, *Partial Differential Equations*, 2nd ed., §2.2.3 (uniqueness via the maximum principle).]

Domain note. The unit square $\Omega = [0,1]^2$ has corners, so "smooth boundary" is technically too strong. The existence and uniqueness result extends to Lipschitz domains (which the square is) via weak solution theory (Evans, Ch. 6), and the solution is C^∞ in the interior. Corner singularities in the gradient can arise if the boundary data has discontinuities at corners, but our corner-consistent BC generation (§2.4) ensures continuous boundary data, so the classical solution exists and the maximum principle applies.

Theorem (Strong Maximum Principle). If T is harmonic on Ω (i.e., $\nabla^2 T = 0$) and not identically constant, then T attains its maximum and minimum on $\partial\Omega$ only. [Exact; Evans, 2nd ed., §2.2.3.]

Consequence for evaluation. The maximum principle provides a physics-based test: for any neural surrogate prediction $\hat{T}$, every interior point must satisfy

$$\min_{\partial\Omega} g \leq \hat{T}(x, y) \leq \max_{\partial\Omega} g \quad \forall (x, y) \in \Omega. \tag{2.1}$$

Violations of (2.1) are unphysical and indicate that the surrogate has not learned the elliptic structure. This is implemented in `src/diffphys/evaluation/physics.py:check_maximum_principle()`.

2.2 Finite-Difference Discretization

We discretize Ω on a uniform grid with spacing $h = 1/(N-1)$, where $N = 64$. Grid points are indexed as $(x_i, y_j) = (ih, jh)$ for $i, j = 0, \ldots, N-1$.

Array indexing convention. Throughout the repository, arrays use the convention $T[i, j] = T(y_i, x_j)$, with axis 0 corresponding to the vertical (y) direction and axis 1 to the horizontal (x) direction. Top-edge values occupy row $i = N-1$; bottom-edge values occupy row $i = 0$; left-edge values occupy column $j = 0$; right-edge values occupy column $j = N-1$.

The 5-point stencil. At each interior point (i, j) with $1 \leq i, j \leq N-2$, the Laplacian is approximated by the standard second-order central difference (LeVeque, 2007, Ch. 3):

$$\nabla^2 T \approx \frac{T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1} - 4T_{i,j}}{h^2} = 0. \tag{2.2}$$

This yields the linear system:

$$L\mathbf{u} = \mathbf{b} \tag{2.3}$$

where $\mathbf{u} \in \mathbb{R}^{(N-2)^2}$ contains the interior unknowns (row-major ordering), L is the sparse $(N-2)^2 \times (N-2)^2$ Laplacian matrix, and $\mathbf{b}$ absorbs the boundary values.

Construction of L. [Exact.] Using row-major indexing $k = i \cdot (N-2) + j$ for the interior grid $(i, j) \in \{0, \ldots, N-3\}^2$, the Laplacian matrix has the block tridiagonal structure:

$$L = \frac{1}{h^2} \begin{pmatrix} B_L & I & & \\ I & B_L & I & \\ & \ddots & \ddots & \ddots \\ & & I & B_L \end{pmatrix} \tag{2.4}$$

where B_L is the $(N-2) \times (N-2)$ tridiagonal matrix

$$B_L = \begin{pmatrix} -4 & 1 & & \\ 1 & -4 & 1 & \\ & \ddots & \ddots & \ddots \\ & & 1 & -4 \end{pmatrix} \tag{2.5}$$

and I is the $(N-2) \times (N-2)$ identity matrix.

Construction of b. The right-hand side vector absorbs boundary values. For interior point (i, j), any neighbor that lies on $\partial\Omega$ contributes $-g_{\text{boundary}}/h^2$ to the corresponding entry of $\mathbf{b}$. Corner and edge-adjacent points pick up contributions from 1–2 boundary neighbors. Interior points far from the boundary have $b_k = 0$.

This is implemented in `src/diffphys/pde/laplace.py:build_laplacian_matrix()` *and* `assemble_rhs()`*. The h^2 factor is absorbed into the matrix for numerical convenience.*

LU factorization and reuse. [Exact.] Since L depends only on the grid size N and not on the boundary data g, we compute the sparse LU factorization $L = P^{-1}L'U'$ once using `scipy.sparse.linalg.splu`. Each subsequent solve requires only forward/backward substitution:

$$\mathbf{u} = L^{-1}\mathbf{b} \quad \text{via lu_factor.solve(b)} \tag{2.6}$$

Cost per solve: $O((N-2)^2)$ flops for back-substitution. On a modern laptop CPU, each solve takes on the order of 0.5ms for $N = 64$; generating 50,000 solutions takes on the order of 25 seconds. (Exact timings are hardware-dependent.)

The solver with LU reuse is implemented in `src/diffphys/pde/laplace.py:LaplaceSolver`*. The factorization is computed once in the constructor. Dataset generation uses* `pde/generate.py`*.*

Truncation error and numerical oracle convention. The 5-point stencil has truncation error $O(h^2)$. For $N = 64$, $h \approx 0.016$, giving truncation error $O(10^{-4})$. For the smooth boundary conditions used in this project, the actual pointwise error versus the analytical solution is well below 10^{-6} (and empirically measured below 10^{-8} for the sinusoidal benchmark in `tests/test_laplace_solver.py`; see §2.3). This threshold should be verified in code for each new BC family — it reflects the smoothness of the specific test solution, not a general property of the stencil at this resolution.

Three distinct numerical quantities should not be confused:

1. **Error vs analytical solution** (§2.3): $\|T_{\text{FD}} - T_{\text{exact}}\|_\infty$. This measures how well the FD discretization approximates the continuum PDE for a specific BC with a known closed-form solution. Empirically $< 10^{-8}$ for the sinusoidal benchmark at $N = 64$.
2. **Discrete Laplacian residual** (§11.2): $R_{\text{PDE}} = \frac{1}{(N-2)^2} \sum r_{i,j}^2$. This measures how well a field satisfies the discretized equation $L\mathbf{u} = \mathbf{b}$. For the FD solver, this is at or near machine precision — empirically measured at $\sim 10^{-10}$ in the benchmark (Table 2), reflecting accumulated floating-point error in the solver and residual computation. For neural surrogates, this is a meaningful physics compliance metric.
3. **Test tolerances**: Automated tests use explicit tolerance thresholds (e.g., $< 10^{-8}$ for analytical match, $\delta = 10^{-6}$ for maximum principle). These are chosen conservatively above the expected numerical precision.

[Convention.] Throughout this repository, "ground truth" means the FD numerical oracle at fixed resolution $N = 64$ — not the exact continuum solution, except where an analytical benchmark is explicitly available (§2.3). All neural surrogate evaluations are against this numerical oracle.

2.3 Analytical Validation

For the boundary condition $T(x, 0) = 0$, $T(x, 1) = \sin(\pi x)$, $T(0, y) = T(1, y) = 0$, the exact solution is:

$$T_{\text{exact}}(x, y) = \sin(\pi x)\frac{\sinh(\pi y)}{\sinh(\pi)} \tag{2.7}$$

[Derivation.] We seek a separable solution $T(x, y) = X(x)Y(y)$. Substituting into (1.1):

$$X''Y + XY'' = 0 \implies \frac{X''}{X} = -\frac{Y''}{Y} = -\lambda \tag{2.8}$$

The x-equation $X'' + \lambda X = 0$ with $X(0) = X(1) = 0$ has eigenvalues $\lambda_n = n^2\pi^2$ and eigenfunctions $X_n = \sin(n\pi x)$. The y-equation $Y'' - n^2\pi^2 Y = 0$ with $Y(0) = 0$ gives $Y_n = \sinh(n\pi y)$. The top boundary condition $T(x, 1) = \sin(\pi x)$ selects $n = 1$ with coefficient $1/\sinh(\pi)$, yielding (2.7). □

The pointwise agreement $\|T_{\text{FD}} - T_{\text{exact}}\|_\infty < 10^{-8}$ at $N = 64$ supports the solver implementation for this benchmark case. This is a necessary but not sufficient validation — it confirms correctness on one analytical solution, not on all possible BCs. Additional tests (zero BCs, symmetric BCs, maximum principle; see §14.1) provide broader coverage. *Validated in* `tests/test_laplace_solver.py`.

2.4 Boundary Condition Generation

The boundary profile g on each of the four edges is generated to produce a diverse training distribution while maintaining corner consistency.

Corner consistency. [Assumption: corners are shared between adjacent edges.] Each corner of Ω is the endpoint of two edges. For the temperature field to be continuous, these edges must agree at corners. We enforce this by:

1. Sampling 4 corner values $c_1, c_2, c_3, c_4 \sim \text{Uniform}(-1, 1)$.
2. Generating each edge profile to interpolate between its two endpoint corners.

Edge profile construction. For an edge parameterized by $x \in [0, 1]$ with corner values c_{start} and c_{end}:

$$g(x) = \underbrace{c_{\text{start}} + (c_{\text{end}} - c_{\text{start}})x}_{\text{linear baseline}} + \underbrace{4x(1-x) \cdot p(x)}_{\text{perturbation}} \tag{2.9}$$

where $p(x)$ is drawn from one of five families (§2.4.1 below). The envelope $4x(1-x)$ forces the perturbation to vanish at $x = 0$ and $x = 1$:

$$4 \cdot 0 \cdot (1-0) \cdot p(0) = 0, \quad 4 \cdot 1 \cdot (1-1) \cdot p(1) = 0 \tag{2.10}$$

guaranteeing $g(0) = c_{\text{start}}$ and $g(1) = c_{\text{end}}$ regardless of p. [Exact.]

The factor 4 normalizes the envelope to have maximum value 1 at $x = 1/2$, so that perturbation amplitudes are interpretable as the maximum deviation from the linear baseline at the edge midpoint.

2.4.1 Perturbation Families Five families of perturbation functions $p(x)$ define the BC diversity:

Family 1 — Sinusoidal:

$$p(x) = A \sin(n\pi x), \quad A \sim \text{Uniform}(0.5, 2.0), \quad n \sim \text{Uniform}\{1, 2, 3, 4\} \tag{2.11}$$

Family 2 — Random Fourier:

$$p(x) = \sum_{k=1}^{K} a_k \sin(k\pi x), \quad a_k \sim \mathcal{N}(0, 1/k^2), \quad K = 5 \tag{2.12}$$

The $1/k^2$ variance decay produces smooth, band-limited profiles. The Karhunen-Loève-like structure ensures that low-frequency modes dominate, yielding physically plausible boundary conditions.

Family 3 — Gaussian bump:

$$p(x) = A \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right), \quad A \sim \text{Uniform}(0.5, 3.0), \quad \mu \sim \text{Uniform}(0.3, 0.7) \tag{2.13}$$

with σ fixed at 0.1. The bump is localized, testing whether surrogates capture local boundary features.

Family 4 — Piecewise constant (held out): Smooth step transitions between 2–3 levels in $[-2, 2]$. This family is **excluded from training** entirely and used only in the OOD test set (`data/test_ood.npz`). It tests whether models generalize beyond the smooth BC families they were trained on.

Family 5 — Linear (no perturbation): $p(x) = 0$. The boundary profile is the linear baseline only. This produces the simplest non-trivial solutions and acts as an easy-case anchor in the training distribution.

Boundary generation is implemented in `src/diffphys/pde/boundary.py`. The held-out split logic is in `pde/generate.py`, controlled by the `allowed_types` parameter.

2.5 The 8-Channel Conditioning Tensor

All neural surrogates receive boundary information through a standardized $(8, 64, 64)$ conditioning tensor. This tensor has two groups of channels:

Value channels (0–3): Each of the four edges (top, bottom, left, right) is encoded as a $(64,)$ profile broadcast into a $(64, 64)$ spatial map: - Channels 0, 1 (top, bottom): profile varies along x-axis, broadcast (tiled) along y-axis. - Channels 2, 3 (left, right): profile varies along y-axis, broadcast (tiled) along x-axis.

Mask channels (4–7): Binary masks indicating which boundary positions are observed vs. interpolated, following the same broadcast convention as the value channels: - Phase 1 (exact BCs): all mask entries are 1.0. - Phase 2 (noisy/sparse BCs): 1.0 at observed positions, 0.0 at interpolated positions.

Design note. The broadcast encoding extends 1D edge information into 2D spatial maps. This is a simple design choice, not a physically exact sensor model — the model must learn to discount the broadcast structure in the interior. A more sophisticated encoding (e.g., attention over sparse observation sets) could improve performance but is not necessary for the benchmark question. The key property is that all models see the same conditioning interface, eliminating architectural confounds.

Mathematically, for the top edge with profile $g_{\text{top}} \in \mathbb{R}^{64}$ and mask $m_{\text{top}} \in \{0, 1\}^{64}$:

$$C_0[i, j] = g_{\text{top}}[j], \quad C_4[i, j] = m_{\text{top}}[j] \quad \forall\, i \in \{0, \dots, 63\} \tag{2.14}$$

and for the left edge with profile $g_{\text{left}} \in \mathbb{R}^{64}$ and mask $m_{\text{left}} \in \{0, 1\}^{64}$:

$$C_2[i, j] = g_{\text{left}}[i], \quad C_6[i, j] = m_{\text{left}}[i] \quad \forall\, j \in \{0, \dots, 63\} \tag{2.15}$$

Implemented in `src/diffphys/data/conditioning.py:encode_bcs()`. *Tested in* `tests/test_conditioning.py`.

---

3. Neural Surrogate Architectures

3.1 U-Net Regressor (Model 1)

The U-Net is an encoder-decoder convolutional network with skip connections (Ronneberger, Fischer & Brox, 2015). We denote the encoder feature maps at resolution level ℓ as $\mathbf{h}_\ell^{\text{enc}}$ and decoder maps as $\mathbf{h}_\ell^{\text{dec}}$.

Architecture. The encoder applies a sequence of residual blocks and spatial downsampling:

$$\mathbf{h}_0^{\text{enc}} = \text{Conv}_{3\times3}(\mathbf{C}; 8 \to 64) \tag{3.1}$$

$$\mathbf{h}_\ell^{\text{enc}} = \text{Down}(\text{ResBlock}(\mathbf{h}_{\ell-1}^{\text{enc}}; c_\ell)), \quad \ell = 1, 2, 3 \tag{3.2}$$

with channel multipliers $(c_1, c_2, c_3, c_4) = (64, 128, 256, 256)$ and spatial resolutions $(64, 32, 16, 8)$. Self-attention is applied at the 16×16 level ($\ell = 2$).

Residual block. Each ResBlock applies two convolutions with GroupNorm (Wu & He, 2018) and SiLU activation:

$$\text{ResBlock}(\mathbf{x}; c) = \mathbf{x} + \text{Conv}(\text{SiLU}(\text{GN}(\text{Conv}(\text{SiLU}(\text{GN}(\mathbf{x}))))) \tag{3.3}$$

with a 1×1 projection on the skip path if the channel count changes.

Bottleneck. At the coarsest resolution (8×8, 256 channels):

$$\mathbf{h}_{\text{bot}} = \text{ResBlock}(\text{Attn}(\text{ResBlock}(\mathbf{h}_3^{\text{enc}}))) \tag{3.4}$$

Decoder. The decoder upsamples and concatenates skip connections:

$$\mathbf{h}_\ell^{\text{dec}} = \text{ResBlock}([\text{Up}(\mathbf{h}_{\ell+1}^{\text{dec}}); \mathbf{h}_\ell^{\text{enc}}]), \quad \ell = 2, 1, 0 \tag{3.5}$$

where $[\cdot\,;\cdot]$ denotes channel-wise concatenation and Up is nearest-neighbor upsampling followed by 3×3 convolution.

Output. A final 3×3 convolution projects to the predicted field:

$$\hat{T} = \text{Conv}_{3\times3}(\mathbf{h}_0^{\text{dec}}; 64 \to 1) \tag{3.6}$$

Loss. Mean squared error against the ground-truth field:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N^2} \sum_{i,j} (\hat{T}_{i,j} - T_{i,j}^{\text{true}})^2 \tag{3.7}$$

Implemented in `src/diffphys/model/unet.py` *(backbone) and* `model/regressor.py` *(MSE wrapper). Trained by* `model/train_regressor.py`*. ~5M parameters.*

3.2 Fourier Neural Operator (Model 2)

The FNO learns an operator mapping between function spaces rather than point-to-point regression. Its key inductive bias is the **spectral convolution layer**, which parameterizes the integral kernel in Fourier space.

Spectral convolution. [Following Li et al. (2021).] For input function $v : \Omega \to \mathbb{R}^{d_v}$, the spectral convolution layer computes:

$$(\mathcal{K}v)(x) = \mathcal{F}^{-1}\big(R_\phi \cdot \mathcal{F}(v)\big)(x) \tag{3.8}$$

where $\mathcal{F}$ denotes the 2D discrete Fourier transform, $R_\phi \in \mathbb{C}^{k_{\max,1} \times k_{\max,2} \times d_v \times d_v}$ is a learnable complex-valued weight tensor (with $k_{\max}$ modes retained in each spatial frequency dimension), and the product truncates to the lowest $k_{\max}$ Fourier modes per axis.

Why truncation works. For smooth PDE solutions (which Laplace solutions are — they are C^∞ in the interior), the Fourier coefficients decay rapidly. Retaining only $k_{\max} = 16$ modes out of 64 captures the dominant spectral content while regularizing the model against high-frequency artifacts. This is a spectral inductive bias aligned with the smoothness of elliptic PDE solutions.

Full FNO layer. Each layer combines the spectral convolution with a pointwise linear transform:

$$v_{\ell+1}(x) = \sigma((\mathcal{K}_\ell v_\ell)(x) + W_\ell v_\ell(x)) \tag{3.9}$$

where $W_\ell \in \mathbb{R}^{d_v \times d_v}$ is a pointwise (i.e., 1×1 convolution) weight matrix and σ is the GeLU activation. The local path $W_\ell v_\ell$ captures high-frequency residuals not represented by the truncated spectral path.

Architecture summary: 1. Lifting layer: $\mathrm{Conv}_{1\times1}(8 \to 32)$ — maps 8-channel conditioning to width-32 latent 2. 4 spectral convolution layers with $k_{\max} = 16$, width 32 3. Projection layer: $\mathrm{Conv}_{1\times1}(32 \to 1)$ — maps to predicted field

Implemented in `src/diffphys/model/fno.py` using the `neuraloperator` library (Kossaifi et al., 2024) or a minimal custom implementation. ~2M parameters.

OOD hypothesis. The FNO's spectral parameterization encodes an inductive bias toward smooth, low-frequency solutions. This may confer an advantage on the held-out piecewise BC family, since Laplace solutions are always smooth in the interior regardless of boundary roughness — the FNO's spectral truncation naturally enforces this smoothness. Whether this advantage materializes is an empirical question tested in Table 6.

Empirical outcome. The FNO significantly underperformed expectations, achieving rel. L2 ≈ 0.40 — roughly 40× worse than the U-Net regressor (rel. L2 ≈ 0.01). This is likely due to insufficient model size or training for this specific problem rather than a fundamental limitation of the FNO architecture. The FNO was not tuned further since the project focus is the generative model comparison; it is included as-is as an operator-learning data point.

3.3 Deep Ensemble (Model 3)

Epistemic uncertainty from seed diversity. A deep ensemble consists of $M = 5$ copies of the U-Net regressor (§3.1), each trained independently with a different random seed s_m:

$$\hat{T}_m = f_{\theta_m}(\mathbf{C}), \quad m = 1, \dots, M \tag{3.10}$$

where θ_m denotes the parameters obtained by training from initialization $\theta_m^{(0)} \sim p(\theta; s_m)$.

Prediction statistics. At inference, the ensemble provides a point estimate and uncertainty:

$$\bar{T}(x, y) = \frac{1}{M} \sum_{m=1}^{M} \hat{T}_m(x, y) \tag{3.11}$$

$$\sigma_T(x, y) = \left(\frac{1}{M-1} \sum_{m=1}^{M} (\hat{T}_m(x, y) - \bar{T}(x, y))^2 \right)^{1/2} \tag{3.12}$$

What the ensemble captures. [Assumption: ensemble variance approximates epistemic uncertainty.] The variance (3.12) reflects disagreement among models that found different solutions due to different initialization and SGD trajectories. Fort, Hu & Lakshminarayanan (2019) showed that this diversity arises because different random initializations explore distinct basins in function space, not merely different weight-space solutions. This captures *epistemic* uncertainty — uncertainty due to finite training data and model capacity — but not *aleatoric* uncertainty arising from the observation noise itself. Ovadia et al. (2019) demonstrated empirically that deep ensembles performed among the strongest UQ methods under dataset shift across their benchmark suite, outperforming MC-Dropout, variational inference, and other approximate Bayesian methods on most tasks.

What the ensemble does not capture. The ensemble does not sample from a posterior distribution over solution fields. Each member produces a deterministic point estimate; the variance arises from initialization diversity, not from a learned generative process. If the 5 members converge to nearly identical predictions, the ensemble uncertainty collapses to near-zero even if the true posterior is broad. This is the fundamental limitation that conditional generative models (flow matching, DDPM) aim to address.

The calibration baseline. If a conditional generative model (FM or improved DDPM) provides better-calibrated uncertainty than the ensemble at matched sample count (5 vs 5), the interpretation is: "the generative model captures richer posterior structure than seed diversity alone." If the ensemble matches the generative models, the interpretation is: "seed diversity suffices for this problem, and the generative machinery adds cost without benefit." The ensemble + conformal wrapper (§9) provides a separate comparison: post-hoc coverage calibration without generative modeling.

CRPS vs coverage caveat. Note that CRPS rewards both calibration *and* sharpness. An ensemble can achieve low CRPS by producing accurate-on-average point predictions with narrow uncertainty bands — even if those bands frequently miss the truth (low coverage). A generative model with slightly higher CRPS but much higher coverage may be the better uncertainty estimator. When interpreting CRPS comparisons, always check coverage and interval width alongside.

Implemented in `src/diffphys/model/ensemble.py`. *Training orchestrated by* `experiments/train_ensemble.py`.

---

4. Score-Based Diffusion and Langevin Dynamics

This section derives the theoretical foundation of DDPM and its connection to score-based generative modeling. The goal is to explain *what* the denoising network learns (the score function), *how* sampling works (discretized Langevin dynamics), and *what the model does not learn* (the PDE itself).

4.1 The Forward Noising Process

Definition. [Exact; Ho et al. (2020).] Given a clean data sample $\mathbf{x}_0 \sim q(\mathbf{x}_0)$, the forward process defines a Markov chain of increasingly noisy versions:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1-\beta_t}\,\mathbf{x}_{t-1},\ \beta_t\mathbf{I}) \tag{4.1}$$

for $t = 1, \dots, T$ with a noise schedule $\beta_1, \dots, \beta_T \in (0, 1)$.

Closed-form marginal. [Exact.] By iterating (4.1), the marginal at any step t is:

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0,\ (1-\bar{\alpha}_t)\mathbf{I}) \tag{4.2}$$

where $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$.

[Derivation of (4.2) from (4.1).] At $t = 1$: $\mathbf{x}_1 = \sqrt{\alpha_1}\,\mathbf{x}_0 + \sqrt{1-\alpha_1}\,\epsilon_1$, with $\epsilon_1 \sim \mathcal{N}(0, \mathbf{I})$. At $t = 2$:

$$\mathbf{x}_2 = \sqrt{\alpha_2}\,\mathbf{x}_1 + \sqrt{1-\alpha_2}\,\epsilon_2 = \sqrt{\alpha_2\alpha_1}\,\mathbf{x}_0 + \sqrt{\alpha_2(1-\alpha_1)}\,\epsilon_1 + \sqrt{1-\alpha_2}\,\epsilon_2 \tag{4.3}$$

The sum of two independent Gaussians with variances $\alpha_2(1-\alpha_1)$ and $(1-\alpha_2)$ gives total variance $\alpha_2 - \alpha_2\alpha_1 + 1 - \alpha_2 = 1 - \alpha_1\alpha_2 = 1 - \bar{\alpha}_2$. By induction, (4.2) holds for all t. □

The reparameterization. Equation (4.2) gives a direct sampling formula:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I}) \tag{4.4}$$

This is what the training loop uses: sample t uniformly, sample ϵ, compute $\mathbf{x}_t$ via (4.4), and train the network to predict ϵ.

Schedule. We use a linear schedule: β_t interpolates linearly from $\beta_1 = 10^{-4}$ to $\beta_T = 0.02$ over $T = 200$ steps. At $t = T$, $\bar{\alpha}_T \approx 0.02$, so $\mathbf{x}_T \approx \mathcal{N}(0, \mathbf{I})$ — the data is nearly destroyed.

4.2 The Reverse Process and Score Function

The reverse diffusion SDE. [Following Song et al. (2021).] The continuous-time limit of the forward process (4.1) is the SDE:

$$d\mathbf{x} = -\frac{1}{2}\beta(t)\,\mathbf{x}\,dt + \sqrt{\beta(t)}\,d\mathbf{w} \tag{4.5}$$

where $\mathbf{w}$ is a standard Wiener process. Anderson (1982) showed that the time-reversal of this SDE is:

$$d\mathbf{x} = \left[-\frac{1}{2}\beta(t)\,\mathbf{x} - \beta(t)\,\nabla_{\mathbf{x}} \log p_t(\mathbf{x})\right] dt + \sqrt{\beta(t)}\,d\bar{\mathbf{w}} \tag{4.6}$$

where $\bar{\mathbf{w}}$ is a reverse-time Wiener process and $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$ is the **score function** — the gradient of the log-density of the noisy data distribution at time t.

Connection to Langevin dynamics. The deterministic part of (4.6) includes a term proportional to the score. This is exactly the drift term in **annealed Langevin dynamics**: at each noise level t, the reverse process follows the score toward high-density regions of p_t, with stochastic noise for exploration. The full reverse trajectory is a noise-annealed MCMC sampler that starts from pure noise ($t = T$) and gradually denoises to a data sample ($t = 0$).

Physical interpretation. The score $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$ points in the direction of steepest ascent of the log-probability at noise level t. At high t (lots of noise), the score provides a coarse, global signal toward the data manifold. At low t (little noise), the score provides fine-grained, local corrections. This multi-scale denoising is why diffusion models can generate diverse, high-quality samples.

4.3 The ϵ-Prediction Objective

DDPM training objective. [Exact; Ho et al. (2020), Eq. 14.] Rather than estimating the score directly, DDPM trains a network $\epsilon_\phi(\mathbf{x}_t, t)$ to predict the noise ϵ added at step t:

$$\mathcal{L}_{\text{DDPM}} = \mathbb{E}_{t,\mathbf{x}_0,\epsilon}\left[\|\epsilon - \epsilon_\phi(\mathbf{x}_t, t)\|^2\right] \tag{4.7}$$

where $t \sim \text{Uniform}\{1, \dots, T\}$, $\mathbf{x}_0 \sim q(\mathbf{x}_0)$, $\epsilon \sim \mathcal{N}(0, \mathbf{I})$, and $\mathbf{x}_t$ is computed via (4.4).

Equivalence to score matching. From (4.2), the conditional score is [Exact]:

$$\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t|\mathbf{x}_0) = -\frac{\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0}{1 - \bar{\alpha}_t} = -\frac{\epsilon}{\sqrt{1 - \bar{\alpha}_t}} \tag{4.8}$$

Therefore, predicting ϵ is equivalent to estimating the score up to a known scaling factor [Approximation: the network ϵ_ϕ approximates the *marginal* score $\nabla \log p_t(\mathbf{x}_t)$, not the conditional score $\nabla \log q(\mathbf{x}_t|\mathbf{x}_0)$ which requires knowing $\mathbf{x}_0$]:

$$\nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t) \approx -\frac{\epsilon_\phi(\mathbf{x}_t, t)}{\sqrt{1 - \bar{\alpha}_t}} \tag{4.9}$$

The DDPM loss (4.7) is a reweighted version of the denoising score matching objective of Vincent (2011). The DDPM ELBO decomposes into per-timestep KL terms, each of which equals a weighted denoising score matching loss (Ho et al., 2020, Eq. 12); predicting ϵ is equivalent to estimating $-\sqrt{1 - \bar{\alpha}_t}\,\nabla_{\mathbf{x}} \log q_t(\mathbf{x})$ via Tweedie's formula. The L_{simple} objective (4.7) drops the ELBO-derived timestep-dependent weights $\beta_t^2/(2\sigma_t^2\alpha_t(1 - \bar{\alpha}_t))$ in favor of uniform weighting over t. This is an intentional design choice: Nichol & Dhariwal (2021) confirmed that uniform weighting produces better sample quality than the ELBO-derived weights. The term "reweighted" refers specifically to this replacement, not to an approximation error.

4.4 Conditional Generation

Conditioning mechanism. In our setting, the DDPM generates solution fields T conditioned on boundary observations $\mathbf{C}$. The network receives the 8-channel conditioning tensor concatenated with the noisy field:

$$\epsilon_\phi(\mathbf{x}_t, \mathbf{C}, t) : \mathbb{R}^{9\times 64\times 64} \times \{1, \dots, T\} \to \mathbb{R}^{1\times 64\times 64} \tag{4.10}$$

The 9 input channels are: 1 channel for $\mathbf{x}_t$ (the noisy field) and 8 channels for $\mathbf{C}$ (the conditioning tensor from §2.5). The timestep t is injected via sinusoidal positional encoding (Vaswani et al., 2017) into the ResBlocks.

What the model learns. The DDPM learns the conditional score $\nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t|\mathbf{C})$ — the gradient of the log-density of solution fields at noise level t, given boundary observations $\mathbf{C}$. For exact boundary conditions, the true conditional distribution over solution fields collapses to a delta measure at the unique PDE solution; accordingly, a well-trained conditional DDPM should approach deterministic behavior, with all samples converging to the same field. In practice, residual sampling variability will persist due to finite model capacity and finite training data — the degree of this variability is itself an indicator of model fit quality. For noisy/sparse observations, the posterior is a genuine distribution over compatible solution fields, and DDPM samples from this posterior.

Critical caveat. The DDPM learns the data distribution $p(\mathbf{x}_0|\mathbf{C})$ induced by the training set, which is generated by the FD solver with a specific prior over boundary profiles (§2.4). It does not learn Laplace's equation in any algebraic sense. If the solver had systematic errors, the DDPM would faithfully reproduce those errors. Physics compliance is an empirical outcome measured by the evaluation metrics (§11), not a property guaranteed by the training objective.

4.5 Sampling (Reverse Process)

DDPM sampling. [Exact; Ho et al. (2020), Algorithm 2.] Starting from $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$, the reverse process iterates:

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_\phi(\mathbf{x}_t, \mathbf{C}, t) \right) + \sigma_t \mathbf{z} \tag{4.11}$$

where $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$ for $t > 1$ and $\mathbf{z} = 0$ for $t = 1$, and $\sigma_t = \sqrt{\beta_t}$.

Derivation of (4.11). [Exact.] The reverse posterior $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ is Gaussian with mean:

$$\tilde{\mu}_t = \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t} \mathbf{x}_0 + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t} \mathbf{x}_t \tag{4.12}$$

Substituting $\mathbf{x}_0 = (\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\epsilon)/\sqrt{\bar{\alpha}_t}$ from (4.4) into (4.12):

$$\tilde{\mu}_t = \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t} \cdot \frac{\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\epsilon}{\sqrt{\bar{\alpha}_t}} + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t} \mathbf{x}_t \tag{4.12a}$$

Collecting the coefficient of $\mathbf{x}_t$:

$$\frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{(1-\bar{\alpha}_t)\sqrt{\bar{\alpha}_t}} + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t} = \frac{\beta_t + \alpha_t(1-\bar{\alpha}_{t-1})}{(1-\bar{\alpha}_t)\sqrt{\alpha_t}} = \frac{1}{\sqrt{\alpha_t}} \tag{4.12b}$$

where we used $\bar{\alpha}_t = \alpha_t\bar{\alpha}_{t-1}$ and $\beta_t + \alpha_t - \alpha_t\bar{\alpha}_{t-1} = 1 - \bar{\alpha}_t$. The coefficient of ϵ simplifies to $-\beta_t/(\sqrt{\alpha_t}\sqrt{1-\bar{\alpha}_t})$. Replacing ϵ with the network prediction ϵ_ϕ yields (4.11).

Denoised estimate. At any step t, the network's prediction implicitly estimates the clean data:

$$\hat{\mathbf{x}}_0^{(t)} = \frac{\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\epsilon_\phi(\mathbf{x}_t, \mathbf{C}, t)}{\sqrt{\bar{\alpha}_t}} \tag{4.13}$$

This denoised estimate is used in the physics regularization (§6). At large t, the estimate is noisy and unreliable. At small t (say $t < T/4$), the estimate is accurate enough to evaluate the PDE residual.

EMA. An exponential moving average of the model weights (Polyak & Juditsky, 1992; Tarvainen & Valpola, 2017) is maintained during training with decay $\gamma = 0.999$:

$$\bar{\theta}_{k+1} = \gamma\bar{\theta}_k + (1-\gamma)\theta_k \tag{4.14}$$

The EMA weights $\bar{\theta}$ are used at inference. This stabilizes sample quality without affecting the training dynamics.

Forward process, reverse sampling, and EMA are implemented in `src/diffphys/model/diffusion.py` *and* `model/sample.py`*. Training loop in* `model/train_ddpm.py`*.*

---

5. Posterior Inference Under Uncertain Observations

5.1 The Observation Model

In Phase 2, the model does not observe the exact boundary profile g. Instead, it receives M noisy point observations per edge:

$$\tilde{g}_i = g(x_i) + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma_{\text{obs}}^2), \quad i = 1, \dots, M \tag{5.1}$$

at positions $x_1, \dots, x_M$ sampled on the edge. More compactly, defining the **observation operator** $\mathcal{H} : T \mapsto (g(x_1), \dots, g(x_M))$ as the boundary trace sampled at the M observation points:

$$\tilde{\mathbf{g}} = \mathcal{H}(T) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(\mathbf{0}, \sigma_{\text{obs}}^2\mathbf{I}_M) \tag{5.1a}$$

The operator $\mathcal{H}$ composes the boundary trace (extracting g from T) with subsampling at the observation positions. The remaining boundary values are reconstructed by linear interpolation between observed points, and the 8-channel mask (§2.5) encodes which positions are directly observed.

What is uncertain. Given sparse noisy observations, the full boundary profile g is unknown. Different boundary completions consistent with (5.1) lead to different PDE solutions. The **posterior** is the distribution over temperature fields T consistent with the observations:

$$p(T|\tilde{\mathbf{g}}) \propto p(\tilde{\mathbf{g}}|T) \cdot p(T) \tag{5.2}$$

where $p(T)$ is the prior over solution fields induced by the BC generation process (§2.4), and $p(\tilde{\mathbf{g}}|T) = \mathcal{N}(\tilde{\mathbf{g}}; \mathcal{H}(T), \sigma_{\text{obs}}^2 \mathbf{I}_M)$ is the Gaussian likelihood from (5.1a).

5.2 Scope of Posterior Claims

[Assumption: synthetic prior.] All posterior evaluations in this project are with respect to the *synthetic* prior over boundary profiles defined by the dataset generator (§2.4) and the observation noise model (5.1). The uncertainty reflects what is unknown given the observations under this specific prior — not a general-purpose physics uncertainty estimate.

This distinction matters. If the boundary prior were different (e.g., drawn from real sensor data, or from a different parametric family), the posterior would be different even with the same observations. The DDPM learns to sample from the data-generating posterior, not from a universal posterior over Laplace solutions.

5.3 DDPM as Approximate Posterior Sampler

When trained on the joint distribution of (observations, solutions), the conditional DDPM learns:

$$\mathbf{x}_0^{(k)} \sim p_\phi(\mathbf{x}_0|\mathbf{C}) \approx p(T|\tilde{\mathbf{g}}), \quad k = 1, \dots, K \tag{5.3}$$

Drawing K independent samples (each from a different noise initialization $\mathbf{x}_T^{(k)} \sim \mathcal{N}(0, \mathbf{I})$) produces an empirical posterior. The sample diversity reflects the model's learned uncertainty: samples should vary in regions where the observations are sparse or noisy, and agree in regions well-constrained by the data.

Comparison with ensemble. The ensemble's 5 predictions also produce a distribution (§3.3), but its variance reflects initialization diversity, not learned posterior structure. In principle, DDPM should produce richer uncertainty because: 1. Each sample is a full reverse-process trajectory, exploring different modes of the posterior. 2. The diversity is learned from data, not from random initialization artifacts.

Whether this theoretical advantage translates to measurably better calibration at $K = 5$ samples is the central empirical question.

5.4 Training for Variable Observation Quality

[Design choice.] Rather than training separate models for each observation regime, a single Phase 2 model handles variable observation quality by randomizing the observation parameters during training:

$$M \sim \text{Uniform}\{8, 12, 16, 24, 32, 48, 64\}, \quad \sigma_{\text{obs}} \sim \text{Uniform}(0, 0.2) \tag{5.4}$$

Each training sample gets a fresh draw of (M, σ_{obs}). The mask channels in the conditioning tensor (§2.5) communicate the observation quality to the model. At evaluation, fixed regimes are used (Table 5 in the implementation plan).

Why from scratch. All Phase 2 models are trained from scratch — no Phase 1 (exact-BC) weights are reused. Warm-starting would give different models different advantages depending on how well exact-BC features transfer to noisy conditioning, muddying the comparison. The from-scratch rule applies equally to the regressor, FNO, ensemble members, DDPM, improved DDPM, and flow matching.

Observation model implemented in `src/diffphys/data/noise_model.py`. Variable-quality training controlled by `data/dataset.py` with `observa/tion_model='random'`.

---

6. Physics-Informed Regularization

Status. This section documents a planned physics-regularized DDPM variant. The derivation, code, and configuration exist in `src/diffphys/model/physics_ddpm.py`, and the regularized loss (Eq. 6.4) is tested in `tests/test_physics_ddpm.py`. **This experiment has not been trained as of this revision.** The method is included in the theory document to describe the intended comparison point for the §11.3 functional CRPS table; the corresponding row is marked "not run" in Tables 3–5. The Jensen's Gap discussion in §6.1 (Zhang & Zou 2025) is the interpretive caveat that would apply if the experiment were run — physics regularization on the denoised mean does not strictly constrain individual samples. Results, if produced, will be added in a future revision.

6.1 Motivation

The standard DDPM objective (4.7) is purely data-driven — it matches the score of the empirical data distribution without any knowledge of the PDE. Adding a physics-informed term asks: does explicit enforcement of $\nabla^2 T = 0$ improve sample quality, and at what cost to sample diversity?

Prior work. Bastek, Sun & Kochmann (ICLR 2025) introduced Physics-Informed Diffusion Models (PIDM), which add PDE residual loss computed on the denoised mean estimate $\hat{\mathbf{x}}_0$ during DDPM training, reducing physics residuals by up to two orders of magnitude. Our approach follows the same principle — computing the PDE residual on $\hat{\mathbf{x}}_0^{(t)}$ from (4.13) — but applies it in the specific context of elliptic PDE surrogates with per-sample timestep weighting.

Jensen's Gap caveat. Zhang & Zou (2025) identified a subtlety in this approach: imposing PDE constraints on $\mathbb{E}[\mathbf{x}_0|\mathbf{x}_t]$ does not strictly constrain individual samples drawn from $p(\mathbf{x}_0|\mathbf{x}_t)$, because $\nabla^2\mathbb{E}[\mathbf{x}_0|\mathbf{x}_t] = 0$ does not imply $\nabla^2\mathbf{x}_0 = 0$ for each sample (Jensen's inequality). For our benchmark, this means physics regularization may improve the *average* Laplacian residual without guaranteeing that *every* sample satisfies the PDE. We report both mean and per-sample residual statistics to assess this.

6.2 The PDE-Residual Loss

At each training step, we compute the denoised estimate (4.13) and evaluate the discrete Laplacian residual on the interior grid:

$$r_{i,j} = \hat{T}_{i+1,j} + \hat{T}_{i-1,j} + \hat{T}_{i,j+1} + \hat{T}_{i,j-1} - 4\hat{T}_{i,j}, \quad 1 \le i, j \le N-3 \tag{6.1}$$

where $\hat{T} = \hat{\mathbf{x}}_0^{(t)}$ from (4.13). The residual field r has shape $(N-2) \times (N-2) = (62, 62)$ for $N = 64$.

The per-sample physics loss:

$$\mathcal{L}_{\text{phys}} = \frac{\sum_{b=1}^{B} w_b \cdot \frac{1}{62^2} \sum_{i,j} r_{i,j}^{(b)\,2}}{\sum_{b=1}^{B} w_b + 10^{-8}} \tag{6.2}$$

where B is the batch size, $r^{(b)}$ is the residual for the b-th sample, and the per-sample weight is:

$$w_b = \mathbb{1}[t_b < T/4] \tag{6.3}$$

6.3 Why Per-Sample Weighting

[Critical implementation note.] The timestep t is sampled independently for each element in the batch: $t_b \sim \text{Uniform}\{1, \dots, T\}$ for $b = 1, \dots, B$.

A naive batch-level check `if max(t_batch) < T/4` would almost never activate. For $T = 200$ and $B = 64$, the probability that all 64 samples have $t < 50$ is $(50/200)^{64} = (1/4)^{64} \approx 10^{-39}$. The physics loss would be effectively absent.

Per-sample weighting (6.3) ensures that approximately 25% of samples per batch contribute to $\mathcal{L}_{\text{phys}}$. Only these low-noise samples produce meaningful denoised estimates — at high t, $\hat{\mathbf{x}}_0^{(t)}$ is dominated by noise and its Laplacian residual is uninformative.

6.4 Combined Objective

$$\mathcal{L} = \mathcal{L}_{\text{DDPM}} + \lambda(e) \cdot \mathcal{L}_{\text{phys}} \tag{6.4}$$

with a warmup schedule:

$$\lambda(e) = \begin{cases} 0 & e < e_{\text{warm}} \\ \lambda_{\max} \cdot \frac{e - e_{\text{warm}}}{e_{\text{ramp}}} & e_{\text{warm}} \le e < e_{\text{warm}} + e_{\text{ramp}} \\ \lambda_{\max} & e \ge e_{\text{warm}} + e_{\text{ramp}} \end{cases} \tag{6.5}$$

with $e_{\text{warm}} = 20$, $e_{\text{ramp}} = 20$, $\lambda_{\max} = 0.1$. The warmup period lets the DDPM learn basic denoising before physics regularization is applied.

What this tests. The physics-regularization experiment asks: does explicit PDE enforcement improve the Laplacian residual of generated samples without degrading CRPS or coverage? If yes, physics regularization adds value. If no (e.g., the data-driven score already captures enough PDE structure), then the base generative model is sufficient. Either finding is informative.

Physics regularization is implemented in `model/physics_ddpm.py` *with the discrete Laplacian computed by* `evaluation/physics.py:discrete_laplacian_batch()`*. Code and config exist but this experiment has not yet been trained — no experimental results are available.*

---

7. Conditional Flow Matching and Optimal Transport

This section derives the flow matching framework as an alternative to DDPM. Where DDPM learns to denoise via a stochastic reverse SDE (§4), flow matching learns a deterministic velocity field that transports noise to data along straight paths. The connection to optimal transport provides a variational principle analogous to action minimization in classical mechanics.

7.1 The Flow Matching Objective

Continuous normalizing flows. A continuous normalizing flow (CNF) defines a time-dependent velocity field $v_t : \mathbb{R}^d \to \mathbb{R}^d$ that generates a probability path p_t from a source distribution $p_0 = \mathcal{N}(0, \mathbf{I})$ to the data distribution $p_1 \approx q(\mathbf{x})$ via the ODE:

$$\frac{d\mathbf{x}}{dt} = v_t(\mathbf{x}) \tag{7.1}$$

The flow matching (FM) objective (Lipman et al., 2023, Eq. 5) regresses a parametric velocity field v_θ against the true generating velocity u_t:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t \sim U(0,1),\, \mathbf{x} \sim p_t} \| v_\theta(\mathbf{x}, t) - u_t(\mathbf{x}) \|^2 \tag{7.2}$$

Computing $p_t(\mathbf{x})$ directly is intractable. The key insight is **conditional flow matching** (CFM): condition on a single data point $\mathbf{x}_1$ and define a tractable conditional probability path $p_t(\mathbf{x}|\mathbf{x}_1)$ with conditional velocity $u_t(\mathbf{x}|\mathbf{x}_1)$. Lipman et al. (2023, Theorem 2) prove that the CFM loss

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t,\, q(\mathbf{x}_1),\, p_t(\mathbf{x}|\mathbf{x}_1)} \|v_\theta(\mathbf{x}, t) - u_t(\mathbf{x}|\mathbf{x}_1)\|^2 \tag{7.3}$$

has identical gradients to (7.2): $\nabla_\theta \mathcal{L}_{\text{FM}} = \nabla_\theta \mathcal{L}_{\text{CFM}}$.

7.2 The Optimal Transport Conditional Path

Gaussian conditional path. Following Lipman et al. (2023, Eqs. 10, 20), the OT-conditional probability path is:

$$p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\big(\mathbf{x};\ t\,\mathbf{x}_1,\ (1 - (1 - \sigma_{\min})t)^2 \mathbf{I}\big) \tag{7.4}$$

As $\sigma_{\min} \to 0$, this produces the **linear interpolant**:

$$\mathbf{x}_t = (1 - t)\,\mathbf{x}_0 + t\,\mathbf{x}_1, \quad \mathbf{x}_0 \sim \mathcal{N}(0, \mathbf{I}),\ \mathbf{x}_1 \sim q(\mathbf{x}) \tag{7.5}$$

The conditional velocity field (Lipman et al., 2023, Eq. 21) simplifies to:

$$u_t(\mathbf{x}|\mathbf{x}_1) = \mathbf{x}_1 - \mathbf{x}_0 \tag{7.6}$$

This velocity is **constant along each conditional path** — independent of t. Contrast with DDPM, where the noise target ϵ enters the loss with timestep-dependent weighting through the schedule $\bar{\alpha}_t$. The constant velocity makes flow matching targets lower-variance and easier to learn.

The CFM training loop. For each training step: sample $\mathbf{x}_1$ from data, $\mathbf{x}_0 \sim \mathcal{N}(0, \mathbf{I})$, $t \sim U(0, 1)$; compute $\mathbf{x}_t$ via (7.5); predict $v_\theta(\mathbf{x}_t, \mathbf{C}, t)$ (with conditioning $\mathbf{C}$); compute MSE loss against $u_t = \mathbf{x}_1 - \mathbf{x}_0$.

Implemented in `src/diffphys/model/flow_matching.py` *and* `model/train_flow_matching.py`*. The U-Net backbone from §3.1 is reused with the same architecture; only the training target and sampling procedure change.*

Conditioning interface. All conditional models (FM, improved DDPM) use the 8-channel conditioning tensor $\mathbf{C}$ from §2.5 (4 value channels + 4 mask channels), giving `in_channels=9` (1 noisy field + 8 condition). The revised Phase 2 implementation plan's code sketches use `in_channels=6`, which reflects an earlier simplified prototype; the implementation should be updated to match the 8-channel reference specification defined here.

7.3 OT-CFM: Mini-Batch Optimal Transport Coupling

Standard CFM pairs noise $\mathbf{x}_0^{(i)}$ with data $\mathbf{x}_1^{(i)}$ by batch index — the coupling is arbitrary. **OT-CFM** (Tong et al., TMLR 2024) replaces this random coupling with a mini-batch optimal transport plan $\pi(\mathbf{x}_0, \mathbf{x}_1)$, minimizing the total transport cost within each batch:

$$\pi^* = \arg \min_{\pi \in \Pi(\hat{p}_0, \hat{p}_1)} \sum_{i,j} \pi_{ij} \|\mathbf{x}_0^{(i)} - \mathbf{x}_1^{(j)}\|^2 \tag{7.7}$$

where $\hat{p}_0, \hat{p}_1$ are the empirical distributions of the noise and data batches, and Π is the set of doubly stochastic coupling matrices. For mini-batches of size B, this is solved exactly via the Hungarian algorithm (`scipy.optimize.linear_sum_assignment`) in $O(B^3)$ — negligible overhead for $B = 64$ on 4096-dimensional data.

Implementation note. Tong et al. (TMLR 2024) use exact OT solvers (Hungarian algorithm) by default. Our implementation matches their default: we compute the pairwise squared-L^2 cost matrix and solve for the exact optimal permutation via `linear_sum_assignment`. This produces exact mini-batch OT couplings, not the Sinkhorn approximation used in their Schrödinger Bridge variant (SB-CFM).

Why straighter flows matter. Tong et al. (TMLR 2024, Proposition 3.4) show that when the true OT plan is available, OT-CFM approximates dynamic optimal transport. Empirically (their Table 2), OT-CFM reduces the Normalized Path Energy (NPE) — a measure of path curvature — approaching the Benamou-Brenier minimum. Straighter flows mean fewer ODE solver steps are needed at inference: 50 Euler steps suffice for 64×64 data, versus DDPM's 200 reverse SDE steps.

OT coupling is implemented in `src/diffphys/model/flow_matching.py:OTCouplingMatche`
Tested in `tests/test_flow_matching.py`.

7.4 Connection to Action Minimization and the Benamou-Brenier Formula

[This section connects the project to the author's mathematical physics background.]

The Benamou-Brenier (2000) dynamic formulation of optimal transport expresses the Wasserstein-2 distance as a variational problem:

$$W_2^2(\mu_0, \mu_1) = \inf_{\rho, v} \int_0^1 \int_{\mathbb{R}^d} \|v(\mathbf{x}, t)\|^2 \, \rho(\mathbf{x}, t) \, d\mathbf{x} \, dt \tag{7.8}$$

subject to the continuity equation $\partial_t \rho + \nabla \cdot (\rho \, v) = 0$ with boundary conditions $\rho(0) = \mu_0$, $\rho(1) = \mu_1$.

The physics parallel. Equation (7.8) has the structure of an **action functional**: the integrand $\|v\|^2 \rho$ is a kinetic energy density, the continuity equation is a constraint, and the infimum selects the minimum-action path between distributions. This is the measure-theoretic analogue of Hamilton's principle $\delta S = \delta \int L \, dt = 0$, where the Lagrangian $L = \frac{1}{2}\|v\|^2$ is purely kinetic (no potential).

In instanton calculus (the author's thesis topic), one minimizes the Euclidean action $S_E[\phi] = \int \frac{1}{2}|\partial_\mu \phi|^2 + V(\phi) \, d^d x$ to find tunneling paths between vacua. The Benamou-Brenier problem is the $V = 0$ analogue: the "instanton" is the geodesic in Wasserstein space connecting two distributions, and OT-CFM approximates this geodesic with learned velocity fields.

McCann's displacement interpolation. The optimal paths in (7.8) are McCann's displacement interpolations: $\rho_t = ((1-t)\,\mathrm{id} + t\, T_*)_{\#}\mu_0$, where T_* is the Monge map. The OT-CFM interpolant (7.5) discretizes this at the sample level. Lipman et al. (2023, §4.1, Example II) connect their OT paths to McCann interpolation; Tong et al. make the Benamou-Brenier connection explicit through their NPE metric.

7.5 Flow Matching vs DDPM: Structural Comparison

Property	DDPM (§4)	Flow Matching (§7)
Generative process	Reverse SDE (stochastic)	ODE (deterministic)
Training target	Noise ϵ	Velocity $\mathbf{x}_1 - \mathbf{x}_0$
Interpolant	$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon$ (nonlinear in t)	$\mathbf{x}_t = (1-t)\,\mathbf{x}_0 + t\,\mathbf{x}_1$ (linear in t)
Target depends on t?	Implicitly (through $\bar{\alpha}_t$ weighting)	No (constant velocity)
Noise schedule needed	Yes (β_t, $\bar{\alpha}_t$ bookkeeping)	No
Sampling steps	200 (SDE)	50 (Euler ODE)
Convergence rate	$O(d)$ (SDE-based)	$O(\sqrt{d})$ (ODE-based, Chen et al., NeurIPS 2023)

The $O(\sqrt{d})$ vs $O(d)$ convergence result for ODE over SDE methods (Chen, Chewi, Lee, Li, Lu & Salim, NeurIPS 2023) provides theoretical motivation for preferring deterministic ODE-based generation. However, this result assumes smooth data distributions with a corrector step and does not directly guarantee that a conditional OT-CFM PDE surrogate will achieve the same dimension-dependent improvement. It is best understood as a directional argument for the ODE framework, not a runtime guarantee for this specific benchmark.

Empirical outcome. On this benchmark, FM's theoretical advantages did not translate into better UQ performance. Improved DDPM (§8) achieved lower functional CRPS on all 5 derived quantities (sparse-noisy regime, matched 5v5) and tighter pixelwise conformal intervals across all observation regimes, despite requiring 4× more sampling steps. On this problem and at this scale, the training improvements (Min-SNR weighting, cosine schedule, v-prediction) appear to have been more impactful than the choice of generative framework. Whether this generalizes to larger-scale or higher-dimensional PDE problems is an open question.

---

8. Improved DDPM Training

This section documents three modifications to the standard DDPM training pipeline (§4) that are motivated by prior work reporting substantially faster and more stable diffusion training. These are applied to produce an improved DDPM as a comparison point against flow matching.

8.1 Cosine Noise Schedule

Motivation. The linear schedule $\beta_t \in [10^{-4}, 0.02]$ (Ho et al., 2020) destroys signal too quickly for low-resolution data. At 64×64, over half the information is lost by $t = T/3$.

Cosine schedule (Nichol & Dhariwal, ICML 2021, Eq. 16):

$$\bar{\alpha}_t = \frac{f(t)}{f(0)}, \quad f(t) = \cos^2\left(\frac{t/T + s}{1+s} \cdot \frac{\pi}{2}\right) \tag{8.1}$$

with $s = 0.008$ chosen so that $\sqrt{\beta_0}$ is slightly smaller than the pixel bin size $1/127.5$. The betas are $\beta_t = 1 - \bar{\alpha}_t/\bar{\alpha}_{t-1}$, clipped to at most 0.999.

8.2 Zero-Terminal SNR

The problem. Common schedules fail to enforce $\bar{\alpha}_T = 0$, leaving residual signal at the final timestep. Lin et al. (WACV 2024, Eq. 10) showed that for Stable Diffusion, $\sqrt{\bar{\alpha}_T} = 0.068$, meaning $\mathbf{x}_T = 0.068\,\mathbf{x}_0 + 0.998\,\epsilon$ — the model sees leaked low-frequency content during training but starts from pure $\mathcal{N}(0, \mathbf{I})$ at inference.

The fix. Rescale the schedule to enforce $\bar{\alpha}_T = 0$ exactly (Lin et al., 2024, Algorithm 1), so $\mathrm{SNR}(T) = \bar{\alpha}_T/(1 - \bar{\alpha}_T) = 0$. Combined with v-prediction (§8.3), this eliminates the training-inference mismatch.

The cosine schedule with zero-terminal SNR is implemented in `src/diffphys/model/diffusion.py:cosine_beta_schedule()`.

8.3 v-Prediction Parameterization

Definition (Salimans & Ho, ICLR 2022, §4). Instead of predicting noise ϵ, the network predicts:

$$v_t = \sqrt{\bar{\alpha}_t} \cdot \epsilon - \sqrt{1 - \bar{\alpha}_t} \cdot \mathbf{x}_0 \tag{8.2}$$

Why v-prediction is more stable. With ϵ-prediction, recovering $\mathbf{x}_0$ requires $\hat{\mathbf{x}}_0 = (\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\hat{\epsilon})/\sqrt{\bar{\alpha}_t}$, which divides by $\sqrt{\bar{\alpha}_t} \to 0$ at high noise — amplifying prediction errors. v-prediction avoids this: $\hat{\mathbf{x}}_0 = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\hat{v}_t$, which remains well-conditioned across all timesteps. The v-prediction loss yields an effective weighting of $1 + \mathrm{SNR}(t)$, which assigns nonzero weight even at $\mathrm{SNR} = 0$ — unlike ϵ-prediction whose weight vanishes there.

8.4 Min-SNR-γ_{SNR} Loss Weighting

The conflict. Different timesteps produce conflicting gradients: low-noise steps push toward fine details while high-noise steps push toward coarse structure. Uniform weighting (as in L_{simple}, Eq. 4.7) lets high-noise steps dominate.

Min-SNR-γ_{SNR} (Hang et al., ICCV 2023, §3.4) clips the signal-to-noise ratio to reduce the contribution of high-noise timesteps (we write γ_{SNR} to distinguish from the EMA decay γ in §4.5):

$$\mathrm{SNR}(t) = \frac{\bar{\alpha}_t}{1 - \bar{\alpha}_t}, \quad w(t) = \frac{\min(\mathrm{SNR}(t),\ \gamma_{\mathrm{SNR}})}{\mathrm{SNR}(t)} \tag{8.3}$$

With $\gamma_{\mathrm{SNR}} = 5$ (the paper's recommended default): timesteps where $\mathrm{SNR} < 5$ receive full weight; timesteps where $\mathrm{SNR} > 5$ are downweighted. The weighted loss is:

$$\mathcal{L}_{\text{MinSNR}} = \mathbb{E}_t\big[w(t) \cdot \|\hat{v}_t - v_t\|^2\big] \tag{8.4}$$

Hang et al. report a **3.4× speedup** in reaching FID 10 compared to previous weighting strategies.

Combined effect. Cosine schedule + zero-terminal SNR + v-prediction + Min-SNR-γ_{SNR} address complementary failure modes of standard DDPM training. Prior work reports up to 3.4× faster convergence from Min-SNR weighting alone (Hang et al., ICCV 2023).

Empirical outcome. The standard DDPM at 60 epochs achieved only 16.8% raw coverage at the 90% target — the loss was still decreasing and the model had not learned the full noise-level spectrum. With the three training improvements, 80 epochs of improved DDPM achieved 85–99% raw coverage across all observation regimes at K=20 samples (the matched K=5 number is 77–95%; see §11.5 for the current primary reporting convention), and the lowest functional CRPS on all 5 derived quantities in the sparse-noisy regime at matched 5-sample count. On this benchmark, the improved DDPM also outperformed flow matching on most metrics despite using 4× more sampling steps. These results are from single training runs (see §16 for replication scope).

All three improvements are implemented in `src/diffphys/model/diffusion.py` *and* `model/train_ddpm.py` *via config flags. Tested in* `tests/test_diffusion.py`.

---

9. Conformal Prediction for Calibrated Uncertainty

This section derives the conformal prediction framework. Standard split conformal prediction provides **distribution-free, finite-sample coverage guarantees**; the pooled-pixel variant used in this project's benchmark is a practical adaptation for field-valued outputs (see exchangeability caveat in §9.1). Unlike DDPM and flow matching, which learn approximate posteriors, conformal prediction wraps any base predictor with a calibration adjustment — at zero additional training cost. The pixelwise variant (§9.1) is the primary method benchmarked in this project; the spatial variant (§9.2) is derived as an additional option for applications requiring simultaneous field-level coverage.

9.1 Pixelwise Split Conformal Prediction

Setup (Vovk et al., 2005; Lei et al., 2018). Given exchangeable calibration data $\{(\mathbf{X}_i, Y_i)\}_{i=1}^n$ and a pre-trained model $\hat{f}$ with uncertainty estimate $\hat{\sigma}$, define the **normalized nonconformity score** (Angelopoulos & Bates, 2021, §2.3.2):

$$R_i = \frac{|Y_i - \hat{f}(\mathbf{X}_i)|}{\hat{\sigma}(\mathbf{X}_i)} \tag{9.1}$$

The conformal quantile is the $\lceil (n+1)(1-\alpha) \rceil / n$-th empirical quantile of $\{R_1, \dots, R_n\}$:

$$\hat{q} = \text{Quantile}\left(\{R_i\}_{i=1}^n,\ \frac{\lceil (n+1)(1-\alpha) \rceil}{n}\right) \tag{9.2}$$

The prediction interval for a new test point $\mathbf{X}_{n+1}$ is:

$$C(\mathbf{X}_{n+1}) = \left[\hat{f}(\mathbf{X}_{n+1}) - \hat{q}\,\hat{\sigma}(\mathbf{X}_{n+1}),\ \ \hat{f}(\mathbf{X}_{n+1}) + \hat{q}\,\hat{\sigma}(\mathbf{X}_{n+1})\right] \tag{9.3}$$

Coverage guarantee (Vovk et al., 2005, Ch. 2; Angelopoulos & Bates, 2021, Theorem 1). For exchangeable data:

$$1-\alpha \ \le\ \mathbb{P}(Y_{n+1} \in C(\mathbf{X}_{n+1})) \ \le\ 1-\alpha+\frac{1}{n+1} \tag{9.4}$$

This is a **finite-sample, distribution-free** guarantee. It holds regardless of the quality of $\hat{f}$ or the distribution of the data — only exchangeability is required. [Exact.]

Pixelwise application to PDE fields. For PDE surrogate outputs, we apply a practical pooled-pixel conformal variant: all $n \times N^2$ pixel-level nonconformity scores are pooled across calibration samples and spatial locations, and the conformal quantile $\hat{q}$ is computed from this pooled set. This is intended to approximate **marginal pixelwise calibration** — producing tighter intervals than the spatial variant (§9.2) — and is the variant used in the benchmark results (Tables 3, 5).

Exchangeability caveat. The textbook split-conformal guarantee (9.4) assumes exchangeable calibration scores. Pooling across spatial locations introduces dependence: pixels within the same field are spatially correlated, so the $n \times N^2$ pooled scores are not fully exchangeable. In practice, the large pool size ($n \times N^2 \gg 1$) and the averaging over many independent calibration fields make the procedure well-behaved empirically, but the formal finite-sample guarantee (9.4) does not strictly apply to this pooled construction. The reported near-nominal 90% coverage across regimes is an empirical observation, not a rigorous theoretical guarantee. Readers seeking rigorous probabilistic guarantees should focus on two things. First, the scalar functional CRPS results in §11.3 are evaluated with a strictly proper scoring rule (Gneiting & Raftery, 2007) and require no exchangeability assumption. Second, standard split conformal applied to those same scalar functionals would yield the finite-sample marginal coverage guarantee of Eq. 9.4 under exchangeability of test points — this is the direct application of §9.1's split conformal procedure to scalar-valued functionals, and is the rigorous pixel-free alternative to the pooled-pixel variant. The pooled-pixel construction reported in Tables 3 and 5 is the pixel-level calibration diagnostic used in this benchmark and should be interpreted empirically.

9.2 Spatial Conformal Prediction (Additional Variant)

For applications requiring coverage at **all spatial locations simultaneously**, motivated by multivariate-output conformal prediction (Feldman et al., JMLR 2023), we define the **spatial maximum** nonconformity score:

$$R_i^{\text{spatial}} = \max_{(m,n)} \frac{|T_i^{\text{true}}(m,n) - \bar{T}_i(m,n)|}{\sigma_i(m,n)} \tag{9.5}$$

where $\bar{T}_i$ and σ_i are the ensemble mean and standard deviation for calibration sample i. Applying split conformal prediction (9.2) to the scalar scores $\{R_i^{\text{spatial}}\}$ yields a prediction band:

$$C_{\text{spatial}}(\mathbf{X}) = \left\{T \,:\, |T(m,n) - \bar{T}(m,n)| \leq \hat{q} \cdot \sigma(m,n) \;\; \forall\, (m,n)\right\} \tag{9.6}$$

with the guarantee $\mathbb{P}(T_{n+1}^{\text{true}} \in C_{\text{spatial}}(\mathbf{X}_{n+1})) \geq 1 - \alpha$, where inclusion means the bound holds at all pixels simultaneously. This guarantee is valid but **conservative by construction**: calibrating to the worst-case pixel produces wider bands than necessary for most spatial locations. Both variants are implemented; the spatial variant was not included in the primary benchmark results.

9.3 Conformal Prediction vs Generative Posterior Sampling

Property	Generative (DDPM / FM)	Conformal (Ensemble + CP)
Coverage guarantee	Approximate (learned)	Exact for standard split CP; pooled-pixel variant here is empirical
Training cost	High (GPU hours)	Zero (post-hoc wrapper)
Interval shape	Implicit (sample diversity)	Explicit $[\mu \pm \hat{q}\sigma]$
Requires calibration set	No	Yes (split from val)
Adapts to observation quality	Via conditioning	Via $\hat{\sigma}$ variation

Complementarity with CRPS. The Ferro (2014) fair CRPS (§11.4) evaluates calibration **post-hoc** as a proper scoring rule. Standard split conformal prediction guarantees coverage **by construction** for exchangeable data; the pooled-pixel variant used here is motivated by that framework but evaluated empirically. They answer different questions: CRPS asks "how well-calibrated is this model's uncertainty?"; conformal prediction says "here is a prediction band calibrated to achieve target coverage."

Important distinction. The conformal guarantee applies to the *prediction band* $C(\mathbf{X})$, not to the *ensemble posterior* itself. If the ensemble's uncertainty estimates $\hat{\sigma}$ are poorly correlated with actual errors, the conformal band will be wide (to compensate); for standard split conformal the coverage guarantee holds, while for the pooled-pixel variant used here coverage is assessed empirically. Conformal prediction fixes the coverage of the output interval; it does not retroactively calibrate the base model's probabilistic predictions.

Empirical outcome (pixelwise conformal, in-distribution, all regimes, matched K=5). Pixelwise conformal prediction lifted all methods to near-nominal 90% coverage across all observation regimes. But the interval widths varied substantially: under sparse-noisy conditions at matched K=5, DDPM pixelwise conformal intervals averaged 0.115, versus 0.156 (FM) and 0.133 (ensemble). This confirms that the raw uncertainty quality — not the conformal wrapper — determines the practical utility of the prediction bands. These results use matched K=5 samples across all methods. At K=20 generative samples (see Appendix Table A1 in benchmark_results.md), the gap widens as expected, but the matched comparison is the primary reported claim. Results are from single training runs.

Implemented in `src/diffphys/evaluation/conformal.py`. *Tested in* `tests/test_conformal.py`.

---

10. Diffusion Posterior Sampling

This section derives the DPS framework for PDE inverse problems. Unlike the conditional models in §4 and §7, which require boundary observations as input during both training and inference, DPS uses an **unconditional** generative prior and incorporates observations via gradient guidance at inference time. This enables zero-shot adaptation to new observation patterns.

10.1 The Inverse Problem Formulation

Given sparse noisy boundary observations $\tilde{\mathbf{g}} = \mathcal{H}(T) + \varepsilon$ from §5.1, the posterior is:

$$p(T|\tilde{\mathbf{g}}) \propto \underbrace{p(T)}_{\text{prior}} \cdot \underbrace{p(\tilde{\mathbf{g}}|T)}_{\text{likelihood}} \tag{10.1}$$

DPS (Chung et al., ICLR 2023) decomposes the posterior score via Bayes' rule:

$$\nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t|\tilde{\mathbf{g}}) = \underbrace{\nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t)}_{\text{unconditional score}} + \underbrace{\nabla_{\mathbf{x}_t} \log p_t(\tilde{\mathbf{g}}|\mathbf{x}_t)}_{\text{likelihood score}} \tag{10.2}$$

The unconditional score is estimated by a diffusion/flow model trained on clean Laplace solutions without boundary condition input. The likelihood score requires marginalizing over $p_t(\mathbf{x}_0|\mathbf{x}_t)$, which is intractable.

10.2 The DPS Approximation

Core approximation (Chung et al., ICLR 2023, Algorithm 1). Replace the intractable marginalization with a point estimate at the Tweedie denoised mean:

$$p_t(\tilde{\mathbf{g}}|\mathbf{x}_t) \approx p(\tilde{\mathbf{g}}|\hat{\mathbf{x}}_0), \quad \hat{\mathbf{x}}_0 = \frac{\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\epsilon_\theta(\mathbf{x}_t, t)}{\sqrt{\bar{\alpha}_t}} \tag{10.3}$$

For Gaussian observations $\tilde{\mathbf{g}}|T \sim \mathcal{N}(\mathcal{H}(T), \sigma_{\text{obs}}^2\mathbf{I})$, the likelihood gradient becomes:

$$\nabla_{\mathbf{x}_t} \log p_t(\tilde{\mathbf{g}}|\mathbf{x}_t) \approx -\frac{1}{\sigma_{\text{obs}}^2} \nabla_{\mathbf{x}_t} \|\tilde{\mathbf{g}} - \mathcal{H}(\hat{\mathbf{x}}_0)\|^2 \tag{10.4}$$

10.3 Dual Guidance: Measurement + Physics

Following DiffusionPDE (Huang et al., NeurIPS 2024, Eq. 8), we extend the DPS update with a **physics guidance** term:

$$\mathbf{x}_{t-1} = \text{reverse_step}(\mathbf{x}_t) - \zeta_{\text{obs}} \nabla_{\mathbf{x}_t} \underbrace{\|\tilde{\mathbf{g}} - \mathcal{H}(\hat{\mathbf{x}}_0)\|^2}_{\mathcal{L}_{\text{obs}}} - \zeta_{\text{pde}} \nabla_{\mathbf{x}_t} \underbrace{\|\nabla^2 \hat{\mathbf{x}}_0\|^2}_{\mathcal{L}_{\text{pde}}} \tag{10.5}$$

The measurement term $\mathcal{L}_{\text{obs}}$ pushes samples toward consistency with the noisy observations. The physics term $\mathcal{L}_{\text{pde}}$ pushes samples toward the Laplace solution manifold. The guidance strengths $\zeta_{\text{obs}}, \zeta_{\text{pde}}$ are annealed during sampling (stronger early, weaker late).

For flow matching, the denoised estimate at ODE step i (out of N steps) is:

$$\hat{\mathbf{x}}_0 \approx \mathbf{x}_t + (1 - t)\, v_\theta(\mathbf{x}_t, t) \tag{10.6}$$

which replaces the DDPM Tweedie estimate (10.3). The guidance gradients (10.4)–(10.5) apply identically.

10.4 Why DPS Matters for Deployment

The key advantage of DPS over conditional models: **a single unconditional model handles any observation pattern without retraining.** If the sensor layout changes — different number of observation points, different noise level, observations on only some edges — the conditional models (§4, §7) must be retrained with the new observation model. DPS adapts at inference time by changing only the observation operator $\mathcal{H}$ in (10.4).

Theoretical status. The original DPS paper (Chung et al., ICLR 2023) provides no formal convergence guarantees — the Tweedie approximation (10.3) is justified empirically. For related unconditional-prior inverse-problem theory, Xu & Chi (NeurIPS 2024) introduced the DPnP framework — a different algorithm that alternates between a proximal consistency sampler and a denoising diffusion sampler — and proved asymptotic consistency under exact scores and diminishing step sizes (their Theorem 1), with non-asymptotic error bounds showing convergence to a distorted posterior π_η with total variation error $O(\eta + \epsilon_{\text{score}} + \epsilon_{\text{solver}})$ (their Theorem 2). These results do not directly apply to the DPS update rule (10.5), but they establish that provably convergent posterior sampling with unconditional diffusion priors is achievable in principle.

DPS is implemented in `src/diffphys/model/dps_sampler.py` *using a trained unconditional DDPM prior (*`src/diffphys/model/unconditional_ddpm.py`*,* `in_channels=1`*). Tested in* `tests/test_dps.py` *and* `tests/test_unconditional_ddpm.p`

10.5 Empirical Outcome

DPS at K=5 demonstrates a clean tradeoff: a ~30× accuracy cost in-distribution is exchanged for robustness to observation patterns the conditional model has never seen. On in-distribution

regimes, DPS reaches the observation noise floor (obs RMSE within 5% of σ_{obs} across 19/20 sparse-noisy examples) while underperforming the conditional DDPM by approximately 30× on relative L^2 error (median 0.056 vs ~0.002 on sparse-noisy). The conditional model's advantage on familiar observation patterns reflects its training-time exposure to the exact observation operator.

Noise floor saturation. DPS saturates the observation noise floor universally, not just on one regime. On dense-noisy (obs RMSE 0.103, $\sigma = 0.1$, ratio 1.03), sparse-noisy (0.095, ratio 0.95), and very-sparse (0.145, $\sigma \approx 0.15$, ratio ≈ 1.0), the observation-space residual matches σ_{obs} within statistical uncertainty. On clean regimes (exact, sparse-clean), the obs RMSE of 0.032 represents the residual measurement-discretization error. DPS has fully exhausted the information content of the measurements in every case.

Physics compliance is regime-invariant. DPS PDE residual spans [4.33, 4.37] across all 5 in-distribution regimes. Because the physics structure comes entirely from the unconditional prior (not from the observation model), PDE compliance is independent of observation quality.

Coverage tracks information-theoretic uncertainty. Coverage at 90% nominal degrades from 96.4% (exact, many high-quality observations) to 56.1% (very-sparse, few noisy observations). This is not a calibration failure — at very-sparse observation density, the true posterior over compatible boundary completions is genuinely wide, and 5 samples may not span it well. The degradation from dense-noisy (93.7%) to sparse-noisy (86.3%) at identical noise levels but different observation counts confirms that coverage tracks the observation information content, not just noise level.

Guidance configuration. Dual guidance (Eq. 10.5) was tuned over a $(\zeta_{\text{obs}}, \zeta_{\text{pde}})$ grid; the best setting had $\zeta_{\text{pde}} = 0$. Any nonzero physics weight ($\zeta_{\text{pde}} \in \{0.001, 0.01, 0.05\}$) caused sampling to diverge to NaN. This is a stronger statement than "physics guidance is unnecessary": on this benchmark, physics guidance is actively destabilizing. The discrete Laplacian gradient through the Tweedie denoised mean amplifies noise at intermediate diffusion steps, consistent with the Jensen's Gap caveat in §6.1. The unconditional prior alone encodes enough elliptic structure that explicit PDE-residual gradients become redundant and harmful. For problems where the prior is less well-matched to the PDE structure (e.g., turbulent flows, nonlinear PDEs), the physics term may become essential.

10.6 Zero-Shot Adaptation

Under zero-shot observation patterns, the in-distribution tradeoff reverses. Tested on three patterns outside the training distribution — extreme noise ($\sigma = 0.5$ vs training max 0.2), non-uniform per-edge sensor density (16/8/32/4 vs training uniform distribution), and single-edge observation (64 points on one edge, zero on the other three) — the conditional model exhibits two qualitatively different failure modes:

Within-manifold degradation. Under extreme noise, the conditional model's PDE residual rises from 4.33 (in-distribution) to 6.84 (1.6×), producing less accurate but still roughly harmonic solutions. DPS produces similar accuracy (rel L^2 0.285 vs 0.298) but better physics compliance (4.62 vs 6.84). At $\sigma = 0.5$, the DPS obs RMSE of 0.424 corresponds to a ratio of 0.85σ — below the noise floor, suggesting the prior provides implicit denoising at higher noise levels. Samples drift toward the prior mean when strict observation matching would be implausibly noisy.

Off-manifold collapse. Under non-uniform sensor density, the conditional model's PDE residual rises from 4.33 to 934 — a 220× blowup indicating samples that no longer approximate Laplace solutions. DPS is unaffected (4.31, within statistical noise of its in-distribution value). On single-edge observation, the conditional model's PDE residual is 41.7 (10× blowup) while DPS remains at 4.26.

The PDE residual separates the two failure modes more sharply than rel L^2. The non-uniform-sensor result is particularly striking: the conditional model has seen both $M = 16$ and $M = 8$ and $M = 32$ and $M = 4$ during training, just never in the joint configuration (16, 8, 32, 4) across the four edges. A 220× increase in PDE residual from this distributional shift suggests the conditional model's generalization is brittle to joint distributional shifts in the observation pattern, even when each marginal shift is in-distribution.

Under single-edge observation, both methods exhibit high rel L^2 (0.926 conditional, 0.848 DPS) because the true posterior over compatible solutions is genuinely wide — one edge provides insufficient information to determine the field. The relevant metric here is not accuracy but physical plausibility: the conditional model's PDE residual of 41.7 indicates it produces solutions that are not approximately harmonic, while DPS at 4.26 produces samples that remain on the Laplace manifold even when the observations are insufficient to localize within it.

DPS, because it accesses observations only through the forward operator $\mathcal{H}$ at inference time, does not encode any implicit joint distribution over per-edge configurations. Its physics compliance is regime-invariant: the PDE residual stays within [4.26, 4.64] across all in-distribution and zero-shot regimes tested. This is the core value proposition of observation-model independence: a single unconditional prior handles sensor configurations that conditional models cannot without retraining.

All DPS results are from a single training run of the unconditional prior and a single tuning pass for $(\zeta_{obs}, \zeta_{pde})$*. No cross-seed variance is reported. The 30× in-distribution gap and zero-shot findings should be interpreted as achievable outcomes, not statistically guaranteed rankings.*

---

11. Evaluation Theory: Proper Scoring and Calibration

This section derives the evaluation metrics used to compare surrogates. The central concepts are **proper scoring rules** (rewarding honest uncertainty) and **calibration** (coverage matching nominal rates). The metrics below are implemented in `src/diffphys/evaluation/metrics.py` and computed by `experiments/eval_phase{1,2}.py`.

11.1 Deterministic Error Metrics

Relative L^2 error. For a predicted field $\hat{T}$ and ground-truth field T:

$$\mathrm{relL}^2(\hat{T}, T) = \frac{\|\hat{T} - T\|_2}{\|T\|_2} = \frac{\sqrt{\sum_{i,j}(\hat{T}_{i,j} - T_{i,j})^2}}{\sqrt{\sum_{i,j} T_{i,j}^2}} \tag{11.1}$$

L^∞ **error.** The worst-case pointwise deviation:

$$\mathrm{L}^\infty(\hat{T}, T) = \max_{i,j} |\hat{T}_{i,j} - T_{i,j}| \tag{11.2}$$

Maximum principle residual. The amount by which a prediction violates the physics constraint (2.1):

$$\text{MP-res}(\hat{T}, g) = \max\left(0,\ \max_{(i,j)\in\Omega^\circ} \hat{T}_{i,j} - \max_{\partial\Omega} g\right) + \max\left(0,\ \min_{\partial\Omega} g - \min_{(i,j)\in\Omega^\circ} \hat{T}_{i,j}\right) \tag{11.3}$$

where Ω° denotes interior grid points. This is zero iff the maximum principle (2.1) holds. A tolerance of $\delta = 10^{-6}$ is used in tests to allow for floating-point error.

11.2 Physics Compliance Metrics

Discrete Laplacian residual. For each interior point, the 5-point stencil residual:

$$R_{\text{PDE}}(\hat{T}) = \frac{1}{(N-2)^2} \sum_{i,j\in\Omega^\circ} \left(\hat{T}_{i+1,j} + \hat{T}_{i-1,j} + \hat{T}_{i,j+1} + \hat{T}_{i,j-1} - 4\hat{T}_{i,j}\right)^2 \tag{11.4}$$

Under the convention that "ground truth" is the FD numerical oracle (§2.2), $R_{\text{PDE}}(T_{\text{oracle}}) \approx 0$. For neural surrogates, R_{PDE} quantifies how well the output satisfies the discretized Laplace equation.

Gradient smoothness. Ensures that sharp artifacts do not appear in the interior:

$$\text{smooth}(\hat{T}) = \frac{1}{N^2} \sum_{i,j} \|\nabla \hat{T}_{i,j}\|^2 \tag{11.5}$$

11.3 Continuous Ranked Probability Score

Definition (Matheson & Winkler, 1976; Gneiting & Raftery, JASA 2007). For a scalar forecast distribution F and observation y:

$$\text{CRPS}(F, y) = \int_{-\infty}^{\infty} \left(F(x) - \mathbb{1}_{x\geq y}\right)^2 dx \tag{11.6}$$

Sample-based estimator. Given K samples $\hat{T}_1, \ldots, \hat{T}_K$ from the forecast distribution and observation T, the empirical CRPS is (Gneiting & Raftery, 2007, Eq. 26):

$$\widehat{\text{CRPS}}(\{\hat{T}_k\}, T) = \frac{1}{K} \sum_{k=1}^{K} |\hat{T}_k - T| - \frac{1}{2K^2} \sum_{k,l} |\hat{T}_k - \hat{T}_l| \tag{11.7}$$

Fair CRPS correction. The estimator (11.7) is biased for small K: because both terms involve averages over the same K samples, the estimated sample-spread (second term) under-represents

the true spread. Ferro (2014) proposed the **fair CRPS** which corrects the second-term denominator from K^2 to $K(K-1)$:

$$\mathrm{CRPS}_{\mathrm{fair}}(\{\hat{T}_k\}, T) = \frac{1}{K}\sum_{k=1}^{K} |\hat{T}_k - T| - \frac{1}{2K(K-1)}\sum_{k\neq l} |\hat{T}_k - \hat{T}_l| \tag{11.8}$$

For $K = 5$, the correction factor is $K/(K-1) = 1.25$ — a 25% upward adjustment of the spread term. This matters: at $K = 5$, the uncorrected CRPS systematically favors models that produce narrower (possibly miscalibrated) distributions. Fair CRPS removes this bias.

Functional CRPS (primary reported metric). In addition to pixel-level CRPS, we compute CRPS on **functional summaries** of the field — the scalar values that a practitioner would care about. For each field T, extract 5 scalar derived quantities: (1) center temperature $T_{\mathrm{center}} = T(N/2, N/2)$, (2) sub-region mean T_{sub}, (3) interior max T_{max}, (4) Dirichlet energy $\int |\nabla T|^2$, (5) top-edge flux $\int_{\mathrm{top}} \partial_n T$. Each functional reduces the field to a single number; we then apply the standard scalar fair CRPS (11.8) to the induced sample distribution.

These functional CRPS values directly answer: "how well is the uncertainty calibrated for the quantities a deployment would actually use?" Pixel-level CRPS averages calibration across all grid points, obscuring whether the model is calibrated where it matters. Functional CRPS is reported as the primary metric for Phase 2 generative model comparisons (§13, Table 3).

11.4 Negative Log-Likelihood

For calibration of ensemble uncertainty, we compute the Gaussian NLL under the ensemble's predicted mean and variance:

$$\mathrm{NLL}(\bar{T}, \sigma_T, T) = \frac{1}{N^2}\sum_{i,j}\left[\frac{1}{2}\log(2\pi\sigma_T^2) + \frac{(T-\bar{T})^2}{2\sigma_T^2}\right] \tag{11.9}$$

NLL is a proper scoring rule (Good, 1952) that penalizes both miscalibrated means and miscalibrated variances. It rewards honest uncertainty: predicting a wide variance when uncertain, a narrow variance when confident, and — critically — not lying about either.

11.5 Coverage Metric

Pixelwise coverage at nominal level $1-\alpha$. For a prediction interval $[\bar{T} - z_\alpha\sigma_T,\ \bar{T} + z_\alpha\sigma_T]$ with $z_{0.05} \approx 1.96$ and true field T:

$$\mathrm{Cov}_{1-\alpha}(\bar{T}, \sigma_T, T) = \frac{1}{N^2}\sum_{i,j}\mathbb{1}\big[|T_{i,j} - \bar{T}_{i,j}| \leq z_\alpha\sigma_{T,i,j}\big] \tag{11.10}$$

Well-calibrated uncertainty produces $\mathrm{Cov}_{0.9} \approx 0.9$. Over-confident models produce $\mathrm{Cov}_{0.9} \ll 0.9$. Over-dispersed models produce $\mathrm{Cov}_{0.9} \gg 0.9$.

Primary sample count convention: matched K=5. For Phase 2 comparisons between generative and non-generative methods, all primary reported CRPS, coverage, and interval-width numbers use $K = 5$ samples across all models. This ensures that generative models are compared to the 5-member ensemble at matched evaluation budget. At $K = 5$, the improved DDPM achieved 77–95% raw coverage across in-distribution observation regimes, versus the ensemble's 15–82% degradation pattern (82% exact → 55% dense-noisy → 31% sparse-noisy → 15% very-sparse). The earlier K=20 numbers cited in §8.4 are retained as a training-improvement baseline but superseded by the matched-K=5 convention for model comparison.

11.6 Matched-Sample-Count Protocol

Why sample count matters for CRPS comparisons. CRPS estimated from more samples is lower (better) than CRPS from fewer samples, because the finite-sample spread term under-represents the true spread. Comparing a DDPM at $K = 50$ samples to an ensemble at $K = 5$ samples is unfair: DDPM gets more samples to estimate the spread, artificially lowering its reported CRPS.

Protocol (primary reporting: matched 5v5). For Phase 2 comparisons:

- **Generative vs ensemble:** 5 DDPM samples vs 5 ensemble predictions (matched with ensemble size).
- **Generative vs generative:** 20 FM samples vs 20 improved-DDPM samples (matched compute budget, different generative frameworks). This comparison is retained in Appendix A1 of `docs/benchmark_results.md` for reference; all primary pixel-level results use matched K=5 per the Current status note below.
- **Spatial conformal prediction:** uses whatever K is needed to estimate σ_T (typically 5 for ensemble, 20 for DDPM), but the comparison is at fixed target coverage $\alpha = 0.1$.

Current status. The primary benchmark comparison reports all generative-vs-ensemble and generative-vs-generative pixel-level and functional metrics at **matched K=5 samples**, following the protocol above. Historical numbers at K=20 (retained in earlier drafts and referenced in §8.4) are preserved as a training-improvement anchor but not used for method-vs-method claims in this revision. All pixel-level CRPS, coverage, and interval-width comparisons (§13 Evidence Mapping rows; Tables 3, 5, 7) use matched K=5.

Fair CRPS is used throughout. All CRPS numbers reported in this document and the benchmark results doc use the Ferro (2014) fair estimator (11.8), not the biased (11.7) estimator. This is a separate correction from sample-count matching.

---

12. Reference Paper Connections

This section maps each reference paper listed in the implementation plan to its specific technical contribution used in this project. For each, I identify what we implement, what we simplify, and what we omit.

12.1 Ho, Jain & Abbeel (NeurIPS 2020) — "Denoising Diffusion Probabilistic Models"

- **We implement:** The forward noising process (4.1), the ϵ-prediction objective (4.7), and the Algorithm 2 reverse sampling loop (4.11). The basic DDPM is our §4 baseline before the improvements in §8.
- **What we use:** Linear β schedule in $[10^{-4}, 0.02]$ (their default) for the baseline; L_{simple} uniform-weight loss (their Eq. 14) for the baseline.
- **What we do not use:** The L_{hybrid} variance-learning objective (their Appendix A) — we use fixed $\sigma_t = \sqrt{\beta_t}$.

12.2 Song, Sohl-Dickstein, Kingma, Kumar, Ermon & Poole (ICLR 2021) — "Score-Based Generative Modeling Through SDEs"

- **We use:** The continuous-time SDE formulation (4.5), the reverse-time SDE with score term (4.6), and the connection between score matching and DDPM. §4.2 of this framework document follows their derivation.
- **What we do not use:** The SDE solvers (Euler-Maruyama, PC). We use the discrete DDPM sampler (4.11).

12.3 Vincent (Neural Computation 2011) — "A Connection Between Score Matching and Denoising Autoencoders"

- **We use:** The denoising score matching objective, which shows that learning to denoise is equivalent to estimating the score of the noisy data distribution. This is the theoretical basis for §4.3's equivalence between ϵ-prediction and score estimation (4.9).

12.4 Fort, Hu & Lakshminarayanan (2019) — "Deep Ensembles: A Loss Landscape Perspective"

- **We use:** Their finding that deep ensemble diversity arises from function-space basin exploration, not just weight-space solutions. This motivates §3.3's claim that seed diversity in a 5-member ensemble provides meaningful epistemic uncertainty.

12.5 Ovadia et al. (NeurIPS 2019) — "Can You Trust Your Model's Uncertainty?"

- **We use:** The empirical result that deep ensembles outperform MC-Dropout, VI, and other approximate Bayesian methods under distribution shift. This supports using ensembles as the calibration baseline in Phase 2.

12.6 Ronneberger, Fischer & Brox (MICCAI 2015) — "U-Net"

- **We implement:** The U-Net encoder-decoder architecture with skip connections (§3.1) as the backbone for the regressor, ensemble, DDPM, and flow matching models.
- **What we modify:** We use residual blocks with GroupNorm (Wu & He, 2018) in place of the original plain convolutions, and add self-attention at the 16×16 resolution level, following the ADM / improved DDPM architectural conventions.

12.7 Wu & He (ECCV 2018) — "Group Normalization"

- **We use:** GroupNorm as the normalization layer in all U-Net residual blocks (§3.1, Eq. 3.3). GroupNorm avoids batch-size dependence of BatchNorm, which is important for small-batch GPU training of large conditional diffusion models.

12.8 Li, Kovachki, Azizzadenesheli, Liu, Bhattacharya, Stuart & Anandkumar (ICLR 2021) — "Fourier Neural Operator for Parametric PDEs"

- **We implement:** The spectral convolution layer (3.8) and the full FNO layer with spectral-plus-local path (3.9). Our FNO uses 4 layers at width 32 with $k_{\max} = 16$ modes.
- **What we do not implement:** The adaptive Fourier neural operator (AFNO) or the tensorized variants. This is a vanilla FNO.

12.9 Kossaifi, Kovachki, Furuya, Baptista, Mukhia, Liu, Kawahara, Anandkumar (Preprint 2024) — "`neuraloperator`"

- **We use:** The `neuraloperator` library as the reference implementation of the FNO, with a custom wrapper to match our 8-channel conditioning interface.

12.10 Ronneberger et al. (MICCAI 2015) → Modern diffusion U-Net (Dhariwal & Nichol, NeurIPS 2021; Peebles & Xie, ICCV 2023) — "Diffusion Architecture"

- **We use:** The modern diffusion U-Net architecture with ResBlock + GroupNorm + SiLU + timestep embedding + self-attention at 16×16 resolution. This is the default architecture for class-conditional image diffusion and is reused here for PDE surrogates. We do not use a Diffusion Transformer (DiT) — the U-Net at this resolution (64×64) is sufficient and computationally efficient.

12.11 Matheson & Winkler (Management Science 1976) — "CRPS"

- **We use:** The definition of CRPS (11.6) as a strictly proper scoring rule for probabilistic forecasts. The sample-based estimator (11.7) is the computational form we use.

12.12 Gneiting & Raftery (JASA 2007) — "Strictly Proper Scoring Rules, Prediction, and Estimation"

- **We use:** The formal result that CRPS is strictly proper (§4.2 of their paper), meaning that the expected CRPS is minimized uniquely by reporting the true predictive distribution. This is the theoretical reason why CRPS is the primary UQ metric.
- **Also from this paper:** The equivalence between CRPS and $\mathbb{E}|Y - Y'| - \frac{1}{2}\mathbb{E}|Y - Y^*|$ for independent copies of the forecast (their Eq. 20), which gives the sample-based estimator.

12.13 Ferro (QJRMS 2014) — "Fair Scores for Ensemble Forecasts"

- **We use:** The fair CRPS correction (11.8) replacing K^2 with $K(K-1)$ in the second-term denominator. This is our primary CRPS metric for all ensemble and generative model eval-

uations.

12.14 Good (JRSS 1952) — "Rational Decisions"

- **We use:** The introduction of the logarithmic scoring rule (NLL), which we use in §11.4 for calibration evaluation of ensemble predictions.

12.15 Lipman, Chen, Ben-Hamu, Nickel & Le (ICLR 2023) — "Flow Matching for Generative Modeling"

- **We implement:** The CFM objective (7.3), the OT-conditional probability path (7.4), and the linear interpolant (7.5). Their Theorem 2 is the theoretical justification for training with conditional rather than marginal flow matching.
- **What we use:** The OT conditional path (their Eq. 20) with $\sigma_{\min} \to 0$, giving the straight-line interpolant.
- **What we do not use:** The variance-preserving (VP) or variance-exploding (VE) conditional paths (their §4.1 Examples III, IV) — we only use the OT path.

12.16 Tong, Fatras, Malkin, Huguet, Zhang, Rector-Brooks, Wolf & Bengio (TMLR 2024) — "Improving and Generalizing Flow-Based Generative Models with Minibatch Optimal Transport"

- **We implement:** OT-CFM with exact mini-batch OT coupling via the Hungarian algorithm (§7.3). We do not implement SB-CFM (their Schrödinger Bridge variant).
- **What we use:** Their Proposition 3.4 (OT-CFM approximates dynamic OT) as the theoretical justification for preferring OT coupling over random coupling.

12.17 Benamou & Brenier (Numerische Mathematik 2000) — "A Computational Fluid Mechanics Solution to the Monge-Kantorovich Mass Transfer Problem"

- **We use:** The dynamic formulation of W_2^2 (7.8) as an action minimization problem. This is the theoretical basis for §7.4's connection to instanton calculus.

12.18 Chen, Chewi, Lee, Li, Lu & Salim (NeurIPS 2023) — "The Probability Flow ODE is Provably Fast"

- **We use:** Their $O(\sqrt{d})$ convergence rate for ODE-based sampling (Theorem 1.1) vs $O(d)$ for SDE-based DDPM (§7.5). This provides asymptotic theoretical motivation for preferring ODE over SDE, though as discussed in §7.5 it does not directly apply to our benchmark setting.

12.19 Salimans & Ho (ICLR 2022) — "Progressive Distillation for Fast Sampling of Diffusion Models"

- **We use:** The v-prediction parameterization (8.2). We do not implement progressive distillation itself (the subject of the paper).

12.20 Nichol & Dhariwal (ICML 2021) — "Improved Denoising Diffusion Probabilistic Models"

- **We implement:** The cosine noise schedule (8.1). We do not implement the other contributions of the paper (learned variance, importance-weighted VLB).

12.21 Hang, Gu, Zhang, Zheng, Chen, Li, Geng, Liang & Guo (ICCV 2023) — "Efficient Diffusion Training via Min-SNR Weighting Strategy"

- **We implement:** Min-SNR-γ_{SNR} weighting (8.3)–(8.4) with $\gamma_{\text{SNR}} = 5$ (their recommended default).

12.22 Lin, Liu, Li & Yang (WACV 2024) — "Common Diffusion Noise Schedules and Sample Steps Are Flawed"

- **We implement:** Zero-terminal SNR (§8.2) via schedule rescaling.

12.23 Bastek, Sun & Kochmann (ICLR 2025) — "Physics-Informed Diffusion Models"

- **We use:** Their core PIDM framework for PDE-aware diffusion training: evaluate PDE residuals on the denoised mean estimate $\hat{\mathbf{x}}_0^{(t)}$ (Eq. 4.13 here) during training, and add the residual loss to the standard DDPM objective. Our §6 physics regularization follows this PIDM structure, with the specific choice of per-sample timestep weighting (6.3) to ensure that low-noise samples contribute consistently.

12.24 Zhang & Zou (Preprint 2025) — "Jensen's Gap in Physics-Informed Diffusion Models"

- **We use:** Their key observation that PDE constraints on the denoised mean $\mathbb{E}[\mathbf{x}_0|\mathbf{x}_t]$ do not strictly constrain individual samples (§6.1), due to Jensen's inequality. This is an essential caveat when interpreting §6's results.

12.25 Vovk, Gammerman & Shafer (2005) — *Algorithmic Learning in a Random World*

- **We use:** The foundational conformal prediction framework and the finite-sample coverage guarantee (9.4) for exchangeable data. Our §9 follows their split conformal construction.

12.26 Angelopoulos & Bates (Monograph 2021) — "A Gentle Introduction to Conformal Prediction"

- **We use:** Their practitioner-oriented exposition of split conformal prediction, in particular the normalized nonconformity score (9.1) and the quantile formula (9.2). Our §9.1 follows their notation.

12.27 Ma, Pitt, Azizzadenesheli & Anandkumar (TMLR 2024) — "Calibrated UQ for Operator Learning via Conformal Prediction"

- **We use:** Their work as motivation for applying conformal prediction to neural operator outputs (§9). Ma et al. propose a risk-controlling quantile neural operator with a functional calibration guarantee on the coverage rate. Our spatial maximum nonconformity score (§9.2, Eq. 9.5) is a different construction — based on the multivariate conformal framework of Feldman et al. (JMLR 2023) — but addresses the same problem: calibrated uncertainty for function-valued predictions.

12.28 Chung, Kim, McCann, Klasky & Ye (ICLR 2023) — "Diffusion Posterior Sampling for General Noisy Inverse Problems"

- **We use:** The DPS approximation (§10.2, Eq. 10.3): replace the intractable likelihood marginalization with a point estimate at the Tweedie denoised mean. The gradient guidance update (Eq. 10.4–10.5) decomposes posterior sampling into prior (unconditional model) + likelihood (measurement + physics gradients).
- **Theoretical status:** No formal convergence guarantees in the original DPS paper. Xu & Chi (NeurIPS 2024) proved asymptotic consistency for a related algorithm (DPnP), establishing that provably convergent posterior sampling with unconditional diffusion priors is achievable. Their results are for DPnP specifically, not for the DPS gradient-guidance update.

12.29 Huang, Liu, Song, Lienen, Jiang, Wilson & Song (NeurIPS 2024) — "DiffusionPDE: Generative PDE-Solving Under Partial Observation"

- **We use:** Dual guidance framework (§10.3, Eq. 10.5) combining measurement loss $\mathcal{L}_{\text{obs}}$ and PDE residual loss $\mathcal{L}_{\text{pde}}$. They train on joint distributions of PDE coefficients and solutions.

12.30 Xu & Chi (NeurIPS 2024) — "Provably Robust Score-Based Diffusion Posterior Sampling for Plug-and-Play Image Reconstruction"

- **We use:** Their DPnP framework as the theoretical reference for convergence-guaranteed unconditional-prior posterior sampling (§10.4). Their Theorem 1 (asymptotic consistency under exact scores and diminishing step sizes) and Theorem 2 (non-asymptotic TV bound) establish that rigorous guarantees are achievable in principle, even though DPS itself lacks them.

---

13. Theory-to-Deliverable Mapping

This section maps the theoretical content of §§2–11 to the deliverables (tables, figures, code) required by the implementation plan. For each deliverable, I identify which theory sections provide the foundation and which metrics are reported.

13.1 Evidence Mapping: Empirical Claims to Sources

Claim	Evidence source	Reproduction command
Sparse-noisy regime: DDPM functional CRPS beats FM and ensemble on Dirichlet energy (0.127 vs 0.276 FM vs 0.190 ens) and top-edge flux (0.044 vs 0.069 FM vs 0.112 ens) at matched K=5	`docs/benchmark_results.md` Table 3	`modal_deploy/evaluate_remote.py --eval-type functional-crps`
Pixelwise conformal mean width (sparse-noisy, matched K=5): DDPM 0.115 < ensemble 0.133 < FM 0.156	`docs/benchmark_results.md` Table 5	`modal_deploy/evaluate_remote.py --eval-type conformal-widths`
Raw ensemble coverage degradation across regimes (82% → 55% → 31% → 15%) at 90% nominal	`docs/benchmark_results.md` Table 4	`modal_deploy/evaluate_remote.py --eval-type coverage-regimes`
Generative coverage at matched K=5: 77–95% across all regimes	`docs/benchmark_results.md` Table 4	`modal_deploy/evaluate_remote.py --eval-type coverage-regimes --k 5`
DPS PDE residual regime-invariance: [4.26, 4.64] across all in-distribution and zero-shot regimes	`docs/benchmark_results.md` Table 7	`modal_deploy/evaluate_remote.py --eval-type dps-regimes`
Conditional DDPM PDE residual blowup under non-uniform sensor configuration: 4.33 → 934 (220×)	`docs/benchmark_results.md` Table 7	`modal_deploy/evaluate_remote.py --eval-type zero-shot`
Conditional DDPM PDE residual blowup under single-edge observation: 4.33 → 41.7 (10×)	`docs/benchmark_results.md` Table 7	`modal_deploy/evaluate_remote.py --eval-type zero-shot`
DPS guidance tuning: $\zeta_{\text{pde}} > 0$ diverges to NaN; best setting has $\zeta_{\text{obs}} = 100, \zeta_{\text{pde}} = 0$	`docs/benchmark_results.md` Appendix B	`modal_deploy/train_dps_guidance.py --grid`
DPS in-distribution accuracy cost: ~30× worse median rel L2 than conditional DDPM at matched K=5 (0.056 vs ~0.002 sparse-noisy)	`docs/benchmark_results.md` Table 3	`modal_deploy/evaluate_remote.py --eval-type dps-accuracy`

Claim	Evidence source	Reproduction command
OOD: DDPM shows lower calibration error than ensemble at matched K=5 across regimes *(FM calibration error not reported in Table 6; DDPM-vs-ensemble claim only)*	`docs/benchmark_results.md` Table 6	`modal_deploy/evaluate_remote.py --eval-type ood-regimes`

13.2 Phase 1 Deliverables

Table 1: Deterministic baselines on exact BCs. Columns: U-Net regressor, FNO, ensemble mean. Rows: rel L2, L∞, MP-res, R_{PDE}. Theory: §3.1, §3.2, §3.3 (architectures), §11.1 (metrics).

Table 2: FD solver validation. Columns: analytical match, boundary consistency, discrete Laplacian residual. Theory: §2.3.

Figure 1: FD solver solutions. 5 example boundary conditions with corresponding solutions. Theory: §2.2, §2.4.

Figure 2: Deterministic baseline errors. Error maps $|\hat{T} - T|$ for each baseline on 5 test cases. Theory: §11.1.

13.3 Phase 2 Deliverables

Table 3: Functional CRPS by method and regime. Columns: center T, sub-region mean, interior max, Dirichlet energy, top-edge flux. Rows: DDPM (K=5), FM (K=5), ensemble (K=5), physics-reg DDPM (not run). Theory: §4, §6, §7, §8, §11.3, §11.6.

Table 4: Raw coverage. Columns: 90% target; rows: ensemble / DDPM / FM, sub-rows: exact, dense-noisy, sparse-noisy, very-sparse. Theory: §3.3 (limitations of ensemble), §5 (posterior), §11.5.

Table 5: Pixelwise conformal widths. Columns: mean interval width at target 90% coverage. Rows: methods × regimes. Theory: §9.1, §11.5.

Table 6: OOD (Family 4 held-out BCs). Columns: rel L2, functional CRPS, coverage. Rows: DDPM, FM, ensemble. Theory: §2.4.1 (Family 4 is held out), §11 (metrics). **FM calibration error is not reported in Table 6; the OOD coverage claim in §13.1 is DDPM-vs-ensemble only.**

Table 7: DPS empirical outcomes. Columns: rel L2, obs RMSE, PDE residual, coverage, coverage/accuracy trade. Rows: DDPM (conditional) vs DPS (unconditional + guidance) across 5 in-distribution regimes and 3 zero-shot regimes. Theory: §10.

Figure 3: Generative sample diversity. 4 samples from each method on same sparse-noisy input, with ground truth. Theory: §5.3.

Figure 4: Conformal interval visualization. Pixelwise interval widths on sparse-noisy test case for DDPM / FM / ensemble. Theory: §9.1.

Figure 5: DPS vs conditional physics compliance. Per-regime PDE residual distributions (log-scale) showing conditional DDPM's 10–220× blowup under zero-shot patterns while DPS stays constant. Theory: §10.5, §10.6.

13.4 What This Framework Supports

This theoretical framework supports the following scientific claims, each with rigor level indicated:

1. **[Rigorous]** The FD solver implementation is correct, with pointwise error $< 10^{-8}$ on the analytical benchmark and discrete Laplacian residual near machine precision.
2. **[Rigorous]** The maximum principle (2.1) and Dirichlet energy decay properties provide physics-based tests independent of the PDE solver.
3. **[Approximate, empirical]** Deep ensembles capture epistemic uncertainty arising from initialization diversity (Fort et al., 2019), but their raw coverage degrades substantially as observations become sparser (82% exact → 15% very-sparse at 90% nominal).
4. **[Approximate, empirical]** Conditional DDPMs trained with modern improvements (cosine schedule, v-prediction, Min-SNR weighting) can achieve better-calibrated posterior samples than deep ensembles for PDE inverse problems at matched K=5 sample counts, with the largest advantages on boundary-sensitive functional quantities.
5. **[Approximate, empirical]** Conditional flow matching (OT-CFM) offers simpler training dynamics but did not outperform improved DDPM on this benchmark — the training improvements were more impactful than the generative framework choice on this problem.
6. **[Empirical, with caveats]** Pixelwise conformal prediction (applied to any base model) lifts raw coverage to near-nominal levels. Standard split conformal prediction has an exact finite-sample coverage guarantee (9.4) for exchangeable calibration scores; the pooled-pixel variant used in this benchmark adapts the method to field-valued outputs and is reported as an empirical diagnostic rather than a rigorous guarantee (see §9.1).
7. **[Empirical]** Conditional DDPM samples have substantially lower relative L^2 errors than DPS on in-distribution observation patterns (~30× at matched K=5), but DPS has regime-invariant physics compliance ([4.26, 4.64] PDE residual) while the conditional model's PDE residual blows up 10–220× under zero-shot observation patterns the conditional model never saw during training.

Claims marked [Approximate, empirical] are contingent on the specific benchmark setup and single-run results. Cross-problem or cross-scale generalization is not established.

---

14. Validation Criteria

This section lists the validation criteria used to verify correctness of each component. These are implemented as automated tests in the `tests/` directory.

14.1 FD Solver Validation

1. **Analytical match.** For the sinusoidal benchmark (2.7), $\|T_{\text{FD}} - T_{\text{exact}}\|_\infty < 10^{-8}$. *Test:* `test_laplace_solver.py::test_sine_benchmark`.
2. **Zero BCs.** If $g = 0$ on $\partial\Omega$, the FD solution should be exactly zero everywhere. *Test:* `test_laplace_solver.py::test_zero_bcs`.
3. **Constant BCs.** If $g = c$ on $\partial\Omega$, the FD solution should equal c at every interior point. *Test:* `test_laplace_solver.py::test_constant_bcs`.
4. **Symmetric BCs.** If g is symmetric under $(x, y) \mapsto (y, x)$, the FD solution should also be symmetric. *Test:* `test_laplace_solver.py::test_symmetric_bcs`.
5. **Maximum principle.** For 10 random BCs, the FD interior maximum should not exceed the boundary maximum (tolerance $\delta = 10^{-6}$). *Test:* `test_laplace_solver.py::test_maximum_principle`.

14.2 Boundary Condition Generation Validation

1. **Corner consistency.** For 100 sampled BCs, adjacent edges agree at corners (difference $< 10^{-10}$). *Test:* `test_boundary.py::test_corner_consistency`.
2. **Family coverage.** The BC generator produces all 5 families with the documented mixture proportions. *Test:* `test_boundary.py::test_family_mixture`.
3. **Perturbation bounds.** Generated BCs stay within $[-3, 3]$ (the valid range after normalization). *Test:* `test_boundary.py::test_bc_range`.
4. **Held-out split.** The training split never contains Family 4 BCs; the OOD test set contains only Family 4 BCs. *Test:* `test_dataset_split.py::test_family4_held_out`.

14.3 Conditioning Tensor Validation

1. **Shape.** The conditioning tensor has shape $(8, 64, 64)$. *Test:* `test_conditioning.py::test_shap`
2. **Value channel encoding.** Channel 0 correctly encodes the top edge profile; channel 2 correctly encodes the left edge profile. *Test:* `test_conditioning.py::test_value_channels`.
3. **Mask channel encoding.** For Phase 1, all masks are 1.0. For Phase 2, masks are 1.0 at observed positions and 0.0 elsewhere. *Test:* `test_conditioning.py::test_mask_channels`.

14.4 DDPM and Diffusion Validation

1. **Forward noising marginals.** At $t = T$, the forward process produces nearly-Gaussian samples: $\|\text{mean}(\mathbf{x}_T) - 0\|_\infty < 0.05$ and $|\text{std}(\mathbf{x}_T) - 1| < 0.05$. *Test:* `test_diffusion.py::test_forward_process_converges`.
2. **Closed-form marginal.** Equation (4.2) agrees with iterated application of (4.1) (up to sampling error). *Test:* `test_diffusion.py::test_marginal_formula`.
3. **Reverse step identity.** For the denoised-mean estimate (4.13), the reverse step formula (4.11) produces the correct posterior mean. *Test:* `test_diffusion.py::test_reverse_poste`
4. **Cosine schedule.** At $t = T$, $\bar{\alpha}_T < 10^{-6}$ (zero-terminal SNR). *Test:* `test_diffusion.py::test_zero_terminal_snr`.
5. **v-prediction.** The v-parameterization (8.2) is numerically stable across all timesteps. *Test:* `test_diffusion.py::test_v_prediction_stability`.

6. **Min-SNR weighting.** The loss weight $w(t)$ is exactly 1 when $\mathrm{SNR}(t) < \gamma_{\mathrm{SNR}}$ and $\gamma_{\mathrm{SNR}}/\mathrm{SNR}(t)$ otherwise. *Test:* `test_diffusion.py::test_min_snr_weighting`.

14.5 Flow Matching Validation

1. **Linear interpolant.** At $t = 0$, $\mathbf{x}_t = \mathbf{x}_0$; at $t = 1$, $\mathbf{x}_t = \mathbf{x}_1$. *Test:* `test_flow_matching.py::test_interpolant_endpoints`.
2. **Constant velocity target.** The target velocity (7.6) is independent of t. *Test:* `test_flow_matching.py::test_target_velocity`.
3. **OT coupling optimality.** The Hungarian algorithm reduces the total transport cost compared to random pairing. *Test:* `test_flow_matching.py::test_ot_coupling_cost`.
4. **Sampling at $t = 1$.** The Euler-integrated ODE produces samples close to the data distribution after 50 steps. *Test:* `test_flow_matching.py::test_sampling_convergence`.

14.6 Conformal Prediction Validation

1. **Coverage on synthetic data.** For a known Gaussian process with known standard deviation, the conformal prediction coverage matches the nominal target (within sampling error). *Test:* `test_conformal.py::test_gaussian_coverage`.
2. **Quantile formula.** The conformal quantile matches the exact formula (9.2). *Test:* `test_conformal.py::test_quantile_formula`.
3. **Interval width monotonicity.** Higher α (lower target coverage) produces narrower intervals. *Test:* `test_conformal.py::test_interval_monotonicity`.

14.7 DPS Validation

1. **Unconditional prior.** The trained unconditional DDPM produces Laplace-solution-like samples without any conditioning input (qualitative check: smooth fields, low PDE residual). *Test:* `test_dps.py::test_unconditional_samples`.
2. **Gradient guidance direction.** For a fixed observation, the gradient guidance term (10.4) points in a direction that decreases the observation residual. *Test:* `test_dps.py::test_guidance_direction`.
3. **DPS sampling stability.** At the tuned $(\zeta_{\mathrm{obs}}, \zeta_{\mathrm{pde}}) = (100, 0)$, the sampling loop does not produce NaN values across 20 test cases. *Test:* `test_dps.py::test_sampling_stability`.

14.8 Metric Validation

1. **CRPS on perfect forecast.** A delta-distribution forecast at the true value produces CRPS = 0. *Test:* `test_metrics.py::test_crps_perfect`.
2. **Fair CRPS correction factor.** For $K = 5$, the fair CRPS spread term is larger than the biased spread term by a factor of $K/(K-1) = 1.25$ when the samples are identical pairs. *Test:* `test_metrics.py::test_fair_crps_correction`.
3. **Coverage on known Gaussian.** For a Gaussian predictive distribution with known σ, the empirical 90% coverage matches the nominal rate (within sampling error). *Test:* `test_metrics.py::test_gaussian_coverage`.

4. **Maximum principle residual.** For a field that respects (2.1), MP-res = 0. For a field with a spike above the boundary maximum, MP-res equals the spike height. *Test:* `test_metrics.py::test_mp_residual`.

---

15. Notation Reference

Symbol	Meaning
$\Omega = [0,1]^2$	Unit square domain
$\partial\Omega$	Boundary of Ω
$N = 64$	Grid resolution
$h = 1/(N-1)$	Grid spacing
$T(x,y)$	Temperature field (solution)
$g(x,y)$	Boundary profile
$\mathcal{H}$	Observation operator (boundary trace + subsampling)
$\tilde{\mathbf{g}}$	Noisy sparse boundary observations
σ_{obs}	Observation noise standard deviation
M	Number of observation points per edge
L	Sparse Laplacian matrix from FD discretization
$\mathbf{C} \in \mathbb{R}^{8\times64\times64}$	8-channel conditioning tensor
$\hat{T}$	Neural surrogate prediction
$\bar{T}, \sigma_T$	Ensemble mean and standard deviation
$\mathbf{x}_0$	Clean data sample (PDE solution)
$\mathbf{x}_t$	Noisy sample at diffusion step t
ϵ	Gaussian noise $\mathcal{N}(0, \mathbf{I})$
$\beta_t, \alpha_t, \bar{\alpha}_t$	DDPM noise schedule parameters
ϵ_ϕ, v_θ	Neural network parameterizations (DDPM / FM)
$\nabla_{\mathbf{x}} \log p_t$	Score function
$\hat{\mathbf{x}}_0^{(t)}$	Tweedie denoised mean estimate
$\text{SNR}(t) = \bar{\alpha}_t/(1-\bar{\alpha}_t)$	Signal-to-noise ratio
γ_{SNR}	Min-SNR clipping threshold (§8.4)
u_t, v_θ	Flow matching velocity fields (true / learned)
W_2^2	Wasserstein-2 distance squared
π^*	Optimal transport coupling
K	Number of samples from generative posterior (or ensemble)
R_i	Conformal nonconformity score
$\hat{q}$	Conformal quantile
$1-\alpha$	Target coverage level (e.g., 0.9)
$\text{CRPS}_{\text{fair}}$	Fair (Ferro 2014) CRPS estimator
$\mathcal{L}_{\text{obs}}, \mathcal{L}_{\text{pde}}$	DPS guidance losses (§10.3)
$\zeta_{\text{obs}}, \zeta_{\text{pde}}$	DPS guidance strengths

16. Bibliography

1. Anderson, B. D. O. (1982). *Reverse-time diffusion equation models.* Stochastic Processes and their Applications, 12(3), 313–326.
2. Angelopoulos, A. N., & Bates, S. (2021). *A gentle introduction to conformal prediction and distribution-free uncertainty quantification.* arXiv:2107.07511.
3. Bastek, J., Sun, W., & Kochmann, D. M. (2025). *Physics-informed diffusion models.* International Conference on Learning Representations (ICLR).
4. Benamou, J.-D., & Brenier, Y. (2000). *A computational fluid mechanics solution to the Monge-Kantorovich mass transfer problem.* Numerische Mathematik, 84(3), 375–393.
5. Chen, S., Chewi, S., Lee, H., Li, Y., Lu, J., & Salim, A. (2023). *The probability flow ODE is provably fast.* Advances in Neural Information Processing Systems (NeurIPS).
6. Chung, H., Kim, J., McCann, M. T., Klasky, M. L., & Ye, J. C. (2023). *Diffusion posterior sampling for general noisy inverse problems.* International Conference on Learning Representations (ICLR).
7. Dhariwal, P., & Nichol, A. (2021). *Diffusion models beat GANs on image synthesis.* Advances in Neural Information Processing Systems (NeurIPS).
8. Evans, L. C. (2010). *Partial differential equations* (2nd ed.). American Mathematical Society.
9. Feldman, S., Bates, S., & Romano, Y. (2023). *Calibrated multiple-output quantile regression with representation learning.* Journal of Machine Learning Research (JMLR).
10. Ferro, C. A. T. (2014). *Fair scores for ensemble forecasts.* Quarterly Journal of the Royal Meteorological Society, 140(683), 1917–1923.
11. Fort, S., Hu, H., & Lakshminarayanan, B. (2019). *Deep ensembles: A loss landscape perspective.* arXiv:1912.02757.
12. Gneiting, T., & Raftery, A. E. (2007). *Strictly proper scoring rules, prediction, and estimation.* Journal of the American Statistical Association, 102(477), 359–378.
13. Good, I. J. (1952). *Rational decisions.* Journal of the Royal Statistical Society, Series B, 14(1), 107–114.
14. Hang, T., Gu, S., Zhang, B., Zheng, J., Chen, Q., Li, M., Geng, Z., Liang, L., & Guo, B. (2023). *Efficient diffusion training via Min-SNR weighting strategy.* International Conference on Computer Vision (ICCV).
15. Ho, J., Jain, A., & Abbeel, P. (2020). *Denoising diffusion probabilistic models.* Advances in Neural Information Processing Systems (NeurIPS).
16. Huang, J., Liu, G., Song, Y., Lienen, M., Jiang, Y., Wilson, A. G., & Song, J. (2024). *DiffusionPDE: Generative PDE-solving under partial observation.* Advances in Neural Information Processing Systems (NeurIPS).
17. Kossaifi, J., Kovachki, N., Furuya, T., Baptista, R., Mukhia, M., Liu, Z., Kawahara, M., & Anandkumar, A. (2024). `neuraloperator`*: A library for learning neural operators.* Preprint.
18. Lei, J., G'Sell, M., Rinaldo, A., Tibshirani, R. J., & Wasserman, L. (2018). *Distribution-free predictive inference for regression.* Journal of the American Statistical Association, 113(523), 1094–1111.

19. LeVeque, R. J. (2007). *Finite difference methods for ordinary and partial differential equations.* SIAM.
20. Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., & Anandkumar, A. (2021). *Fourier neural operator for parametric partial differential equations.* International Conference on Learning Representations (ICLR).
21. Lin, S., Liu, B., Li, J., & Yang, X. (2024). *Common diffusion noise schedules and sample steps are flawed.* Winter Conference on Applications of Computer Vision (WACV).
22. Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., & Le, M. (2023). *Flow matching for generative modeling.* International Conference on Learning Representations (ICLR).
23. Ma, Z., Pitt, J., Azizzadenesheli, K., & Anandkumar, A. (2024). *Calibrated uncertainty quantification for operator learning via conformal prediction.* Transactions on Machine Learning Research (TMLR).
24. Matheson, J. E., & Winkler, R. L. (1976). *Scoring rules for continuous probability distributions.* Management Science, 22(10), 1087–1096.
25. Nichol, A. Q., & Dhariwal, P. (2021). *Improved denoising diffusion probabilistic models.* International Conference on Machine Learning (ICML).
26. Ovadia, Y., Fertig, E., Ren, J., Nado, Z., Sculley, D., Nowozin, S., Dillon, J., Lakshminarayanan, B., & Snoek, J. (2019). *Can you trust your model's uncertainty? Evaluating predictive uncertainty under dataset shift.* Advances in Neural Information Processing Systems (NeurIPS).
27. Peebles, W., & Xie, S. (2023). *Scalable diffusion models with transformers.* International Conference on Computer Vision (ICCV).
28. Polyak, B. T., & Juditsky, A. B. (1992). *Acceleration of stochastic approximation by averaging.* SIAM Journal on Control and Optimization, 30(4), 838–855.
29. Ronneberger, O., Fischer, P., & Brox, T. (2015). *U-Net: Convolutional networks for biomedical image segmentation.* Medical Image Computing and Computer-Assisted Intervention (MICCAI).
30. Salimans, T., & Ho, J. (2022). *Progressive distillation for fast sampling of diffusion models.* International Conference on Learning Representations (ICLR).
31. Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., & Poole, B. (2021). *Score-based generative modeling through stochastic differential equations.* International Conference on Learning Representations (ICLR).
32. Tarvainen, A., & Valpola, H. (2017). *Mean teachers are better role models.* Advances in Neural Information Processing Systems (NeurIPS).
33. Tong, A., Fatras, K., Malkin, N., Huguet, G., Zhang, Y., Rector-Brooks, J., Wolf, G., & Bengio, Y. (2024). *Improving and generalizing flow-based generative models with minibatch optimal transport.* Transactions on Machine Learning Research (TMLR).
34. Vincent, P. (2011). *A connection between score matching and denoising autoencoders.* Neural Computation, 23(7), 1661–1674.
35. Vovk, V., Gammerman, A., & Shafer, G. (2005). *Algorithmic learning in a random world.* Springer.
36. Wu, Y., & He, K. (2018). *Group normalization.* European Conference on Computer Vision (ECCV).
37. Xu, X., & Chi, Y. (2024). *Provably robust score-based diffusion posterior sampling for plug-and-play image reconstruction.* Advances in Neural Information Processing Systems

(NeurIPS).
38. Zhang, T., & Zou, D. (2025). *Jensen's gap in physics-informed diffusion models.* Preprint.

Replication scope. All empirical numbers in this document come from single training runs of each method on single data splits. No seed averaging or statistical significance tests have been performed. DPS results derive from a single unconditional-prior training run and a single guidance-tuning pass over the $(\zeta_{\text{obs}}, \zeta_{\text{pde}})$ grid. Quantitative rankings should be interpreted as achievable outcomes on this specific benchmark configuration, not as statistically guaranteed orderings. The primary design choices — fixed resolution $N = 64$, smooth synthetic boundary prior (§2.4), Laplace equation rather than a harder PDE — further scope the generalizability of these findings.

Contact. Jane Yeung. `diffphys` package at `github.com/tyy0811/laplace-uq-bench`. Issues and replication queries welcome.